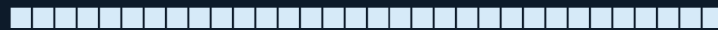


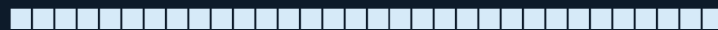
■ JAVA MICROSERVICES

Spring Cloud — Complete Professional Study Notes



FULL THEORY • DEEP EXPLANATIONS • CODE EXAMPLES

ALL QUESTIONS WITH COMPLETE ANSWERS



Beginner → Senior Engineer Level

The Explora Specialization | Jan 2026 Cohort

5 Chapters **110+** Questions **110+** Answers **500+** Score

- ✓ Chapter 01: Introduction to Microservices — Architecture, Patterns, 12-Factor App
- ✓ Chapter 02: Eureka Server — Service Registry, Discovery, Heartbeat, HA
- ✓ Chapter 03: Feign Client — Declarative REST, Error Handling, Interceptors
- ✓ Chapter 04: Spring Cloud Gateway — Routing, Filters, Auth, Rate Limiting
- ✓ Chapter 05: Spring Cloud Load Balancer — Strategies, Integration, Resilience

★ EVERY question answered with full theory, diagrams, and code examples

Crafted with 20+ Years of Engineering Perspective — For Freshers Who Want Senior-Level Understanding

TABLE OF CONTENTS

01 Introduction to Microservices

- › 1.1 What Are Microservices?
- › 1.2 Monolith vs Microservices
- › 1.3 Core Principles (SRP, Loose Coupling, High Cohesion...)
- › 1.4 Benefits — Independence, Scalability, Resilience
- › 1.5 Challenges — Distributed Complexity, Data, Ops
- › 1.6 The 12-Factor App — All 12 Factors Explained
- › 1.7 When to Use Microservices — Decision Framework
- › Practice Questions 1–22 with Full Answers

02 Service Registry & Discovery — Eureka Server

- › 2.1 The Problem: Why Service Discovery?
- › 2.2 How Eureka Works — Registration, Heartbeat, Fetch
- › 2.3 Self-Preservation Mode — Deep Explanation
- › 2.4 Eureka Server Setup — Full Configuration
- › 2.5 Eureka Client — Registration Configuration
- › 2.6 High Availability Eureka Cluster
- › Practice Questions 1–22 with Full Answers

03 Feign Client — Inter-Service Communication

- › 3.1 The Problem with RestTemplate
- › 3.2 What Is Feign? How It Works Internally
- › 3.3 @FeignClient — All Parameters Explained
- › 3.4 Error Handling — ErrorDecoder, Fallback
- › 3.5 Feign Configuration — Timeout, Logging, Retry
- › 3.6 RequestInterceptor — JWT Propagation
- › Practice Questions 1–22 with Full Answers

04 API Gateway — Spring Cloud Gateway

- › 4.1 Why API Gateway Exists — Problems It Solves
- › 4.2 Route, Predicate, Filter — Core Concepts
- › 4.3 Complete YAML Route Configuration
- › 4.4 Global Filters — Logging, Auth, CORS
- › 4.5 Rate Limiting with Redis
- › 4.6 Circuit Breaker at Gateway
- › Practice Questions 1–22 with Full Answers

- › 5.1 What Is Load Balancing?
- › 5.2 Client-Side vs Server-Side
- › 5.3 Round Robin, Random, Zone-Aware Strategies
- › 5.4 @LoadBalanced — RestTemplate & WebClient
- › 5.5 Feign + LoadBalancer Integration
- › 5.6 Custom Load Balancing Strategy
- › Practice Questions 1–22 with Full Answers

Introduction to Microservices

The architectural revolution that changed how the world builds software.

1.1 What Are Microservices?

Microservices is an **architectural style** in which a large application is decomposed into a collection of small, autonomous, independently deployable services. Each service is responsible for a **specific business capability**, runs in its own process, and communicates with other services through **lightweight mechanisms**, typically HTTP/REST APIs or message queues.

The term was formally defined by **Martin Fowler and James Lewis** in their 2014 article. However, the concept had been practiced at companies like Amazon and Netflix years before it had a name.

Key characteristics of a true microservice:

- **Single Business Capability:** A service does ONE business thing. OrderService manages orders. UserService manages users. PaymentService handles payments. There is zero overlap.
- **Own Process:** Each service runs as a separate JVM process (or container). Crashing one does not crash others.
- **Own Database:** The 'Database per Service' pattern — each service owns its data. OrderService has its own MySQL DB. UserService has its own PostgreSQL DB. They do NOT share tables.
- **Lightweight Communication:** Services talk via REST APIs, gRPC, or async messaging (Kafka, RabbitMQ). No shared memory, no direct method calls.
- **Independent Deployment:** Deploy OrderService without touching UserService. Zero downtime releases possible.
- **Independently Scalable:** Scale only the service that's bottlenecked. If PaymentService is slow, spin up 10 instances of it, leave others untouched.
- **Technology Agnostic:** OrderService in Java, RecommendationService in Python, NotificationService in Node.js — all communicating via REST.

■ ANALOGY

A microservices system is like a modern hospital. There's a Cardiology department, Neurology department, Pharmacy, Lab, and Radiology. Each department (service) is specialized, has its own staff and equipment (data), operates independently, but they communicate via referral forms (APIs). If the Radiology department is overwhelmed, you add more radiologists (scale that service), not more cardiologists.

1.2 Monolith vs Microservices

1.2.1 The Monolithic Architecture

A monolith is an application where ALL components — presentation layer, business logic, data access layer — are bundled into a **single deployable artifact** (WAR or JAR file). Every change requires rebuilding and redeploying the entire application.

The lifecycle of a typical monolith goes like this: Early on (1–20 developers), the monolith is actually quite productive. Code sharing is easy, no network calls, no distributed systems problems. But as the team grows to 50, 100, 200 developers all working on one codebase, things deteriorate rapidly.

Problems that emerge with monolith at scale:

- **The Deployment Problem:** A team of 200 developers makes 50 PRs per day. Every PR must be merged, tested, and the entire app redeployed. One bad PR blocks everyone.
- **The Scaling Problem:** Your checkout flow handles 1,000 req/sec during a sale. But to scale it, you must scale the ENTIRE application — including the admin dashboard that handles 1 req/day. Massive waste of resources.
- **The Technology Debt Problem:** You're locked into Java 8 because the whole app runs on it. Upgrading means testing EVERYTHING. It never gets done.
- **The Fault Isolation Problem:** A memory leak in the reporting module OOMs the JVM and takes down login, checkout, and everything else.
- **The Team Coordination Problem:** Team A wants to refactor the User table schema. This breaks Team B's Order module and Team C's Report module. Weeks of coordination needed.

1.2.2 Detailed Comparison

Dimension	Monolithic	Microservices
Codebase	Single repo, all in one	Multiple repos, one per service
Deployment	One artifact — deploy all or nothing	Deploy each service independently
Scaling	Scale entire app	Scale individual services
Technology	Locked into one stack	Polyglot — any language per service
Database	Single shared database	Database per service
Fault Impact	One bug can crash everything	Failures isolated per service
Team Size	Works for small teams	Scales with large teams

Dev Complexity	Simple early, complex at scale	Complex early, manageable at scale
Ops Complexity	Simple (one thing to deploy)	Complex (50+ things to manage)
Performance	In-process calls (0ms overhead)	Network calls (1-10ms overhead)
Testing	Easy to do integration tests	Requires contract testing
Initial Cost	Low	High (infrastructure investment)

1.3 Core Principles of Microservices

◆ Single Responsibility Principle (SRP)

This is borrowed from object-oriented design (Robert C. Martin). In microservices, it means each service has ONE reason to change. If your service changes because of business logic changes AND because of database schema changes AND because of UI changes, it's doing too much. Example: OrderService should only change when order-related business rules change. Not when user authentication rules change. Not when payment rules change.

◆ Loose Coupling

Services should be as independent as possible. Loose coupling means: changing Service A does not require changing Service B. Achieved through: stable API contracts (don't break existing endpoints), asynchronous communication (events), and no shared database. Anti-pattern: Service A imports Service B's Java classes directly — this is tight coupling and defeats the purpose of microservices.

◆ High Cohesion

Everything related to one domain should live in one service. All order-related logic, all order-related database tables, all order-related business rules live in OrderService. High cohesion makes each service self-contained and easier to understand. Anti-pattern: OrderService calling UserService to do order validation — that logic should be in OrderService.

◆ Decentralized Governance

No single team dictates technology choices for all services. Each team can choose: Java vs Python vs Node.js, MySQL vs MongoDB vs Redis, gRPC vs REST vs GraphQL. They're only constrained by the API contract (how they expose their service). This enables innovation and teams can use the right tool for the job.

◆ Decentralized Data Management

No shared database. Each service owns its data. This is the most controversial principle because it makes transactions hard. But it's essential for independence. If Service A and B share a database, changing a table schema in Service A breaks Service B. With separate databases, each team manages their own schema independently.

◆ Failure Isolation (Design for Failure)

In a distributed system, failures WILL happen. A network call can fail. A service can crash. A database can be slow. Good microservices design assumes failures and handles them gracefully. Use circuit breakers (Resilience4j), timeouts, retries, and fallbacks. If RecommendationService is down, show popular products — don't crash the homepage.

◆ Infrastructure Automation

With 50+ services, manual deployment is impossible. You NEED: CI/CD pipelines (Jenkins, GitHub Actions), containerization (Docker), orchestration (Kubernetes), infrastructure as code (Terraform). This is NOT optional — it's a prerequisite.

1.4 Benefits — In Depth

✓ Independent Scalability

This is the most compelling reason for e-commerce companies. Amazon's Checkout Service needs 100x capacity on Prime Day. In a monolith, you'd scale everything 100x at enormous cost. With microservices, only CheckoutService scales. The ProductCatalogService, which handles read-only traffic efficiently, stays at 5 instances. Cost savings: 80%.

✓ Faster Development Velocity

With 200 developers on a monolith, merge conflicts are daily occurrences. Feature branches live for weeks. With microservices, Team A (UserService) and Team B (OrderService) work in parallel, deploy independently, and never conflict. Amazon reportedly deploys code every 11.6 seconds — only possible with microservices.

✓ Technology Flexibility (Polyglot)

Netflix uses Java for its core API, Python for its recommendation ML models, and JavaScript for its UI services — all in the same ecosystem. When Go becomes superior for a specific use case, they can adopt it in one service without migrating the whole platform.

✓ Fault Isolation

Netflix perfected this with the concept of 'Chaos Engineering' (Chaos Monkey). They deliberately kill services in production to ensure the system degrades gracefully. When recommendations fail, users still see content. When subtitles service fails, video still plays. Each service failure has zero impact on unrelated services.

✓ Easier to Understand and Maintain

A 10,000-line service with clear boundaries is FAR easier to understand than a 1,000,000-line monolith. New engineers become productive in one service quickly. The codebase is small enough to hold in your head.

1.5 Challenges — The Real Costs

No senior engineer will recommend microservices without acknowledging these severe challenges:

■ Distributed Systems Complexity

When you move from method calls to network calls, you inherit ALL problems of distributed computing. The 'Eight Fallacies of Distributed Computing' (Peter Deutsch) say: the network

is NOT reliable, latency is NOT zero, bandwidth is NOT infinite, the network is NOT secure, topology DOES change, there are MULTIPLE administrators, transport cost is NOT zero, the network is NOT homogeneous. Every one of these creates bugs you'd never see in a monolith.

■ Data Consistency

Without a shared database, you lose ACID transactions. If OrderService deducts inventory and PaymentService processes payment — what if payment fails AFTER inventory is deducted? In a monolith: rollback the transaction. In microservices: you need the Saga Pattern, which is complex. Either Choreography (events) or Orchestration (saga coordinator).

■ Service Discovery

How does ServiceA find ServiceB when it can be on any machine, any port, with any number of instances? You need infrastructure like Eureka, Consul, or Kubernetes Service DNS. This is non-trivial to set up and operate.

■ Operational Overhead

With 50 services, you need: 50 CI/CD pipelines, 50 sets of logs to search, 50 health monitors, distributed tracing (Jaeger/Zipkin), centralized log aggregation (ELK Stack), metrics dashboards (Prometheus + Grafana), and a team of SREs/DevOps engineers to maintain all of this. The operational maturity requirement is HIGH.

■ Testing Complexity

Integration testing in a monolith is easy — spin up one app, run tests. In microservices: which services do you spin up? All 50? That's expensive. Use Consumer-Driven Contract Testing (Pact), service mocks, and separate integration test environments.

■ Network Latency

A checkout flow that calls: UserService (2ms) + OrderService (3ms) + InventoryService (2ms) + PaymentService (5ms) + NotificationService (2ms) = 14ms minimum just in network overhead. In a monolith: 0ms (method calls). Design APIs to minimize round trips. Use caching aggressively.

X MISTAKE

The biggest mistake freshers make: assuming microservices are 'better' than monoliths. They're not better — they're DIFFERENT and solve DIFFERENT problems. For a startup with 5 developers, microservices are an over-engineered disaster. For Netflix with 2,000 engineers, a monolith is unmanageable.

1.6 The 12-Factor App

The 12-Factor App is a methodology for building software-as-a-service (SaaS) apps that are cloud-native, portable, and scalable. Developed by engineers at Heroku. Every Spring Boot microservice should follow these factors:

■ I. Codebase — One Codebase, Many Deploys

Each service has exactly ONE codebase in version control. That one codebase produces multiple deploys: development, staging, production. VIOLATION: Two services sharing the

same repository and being deployed together — they should be separate.

■ II. Dependencies — Declare and Isolate

All dependencies must be explicitly declared in `pom.xml` or `build.gradle`. Never rely on system tools being installed (e.g., `'curl'` being available on the server). Spring Boot's embedded Tomcat is a good example — the HTTP server is declared as a dependency, not assumed to exist on the server.

■ III. Config — Store Config in Environment

NEVER hardcode configuration in code or commit it to version control. Database URLs, API keys, passwords must come from environment variables. In Spring Boot: `application.properties` can reference env vars: `spring.datasource.url=${DB_URL}`. Use Spring Cloud Config Server for centralized config.

■ IV. Backing Services — Treat as Attached Resources

Databases, message queues, caches should be 'attached resources' configurable via URL. You should be able to swap from local MySQL to AWS RDS by changing a config property, not code. Your service should NOT know or care WHERE the database is physically located.

■ V. Build, Release, Run — Strict Separation

Build stage: compile code + resolve dependencies → produce artifact (JAR/Docker image). Release stage: combine artifact with config for specific environment. Run stage: execute the release. NEVER modify code at runtime. NEVER deploy by SSHing into a server and changing files.

■ VI. Processes — Stateless

Services must be stateless. No sticky sessions. No in-memory state that can't be recreated. If you store user sessions in memory and the instance crashes, those users lose their session. Store sessions in Redis. Store files in S3. This enables horizontal scaling: just add more instances.

■ VII. Port Binding — Export Services via Port

The service is self-contained and exposes itself via a port. Spring Boot embeds Tomcat and binds to port 8080. No external web server (Apache/Nginx) needed. This makes the service runnable anywhere: locally, in Docker, on any cloud.

■ VIII. Concurrency — Scale via Process Model

Scale out by running more instances (processes), not by making one instance bigger (threads). Don't increase heap size — add more JVM instances. This maps naturally to Kubernetes pods.

■ IX. Disposability — Fast Startup, Graceful Shutdown

Services should start in seconds, not minutes. On `SIGTERM`, complete in-flight requests and shut down. Spring Boot Actuator provides graceful shutdown support. This enables fast deployments, auto-scaling, and self-healing.

■ X. Dev/Prod Parity — Keep Environments Similar

The 'works on my machine' problem. Use Docker to make dev and prod identical. Same database version, same OS, same Java version everywhere. The further dev diverges from

prod, the more bugs appear only in production.

■ XI. Logs — Treat as Event Streams

Services should NOT manage log files. Write logs to stdout. Let the platform (Docker, Kubernetes) collect them. An aggregator (ELK Stack, Splunk, CloudWatch) stores and searches them. Never do: new File('app.log') in a microservice.

■ XII. Admin Processes — One-off Tasks

Database migrations, data cleanup scripts, admin tasks should run as one-off processes in the same environment as the app. Use Flyway/Liquibase for DB migrations. Never run migrations inside the app's startup code — this causes issues with multiple instances.

✓ TIP

Spring Boot microservices naturally implement many 12-Factor principles: Factor VII (port binding via embedded Tomcat), Factor IX (graceful shutdown via Actuator), Factor XI (SLF4J logs to stdout). Focus on explicitly implementing III (Config) via Spring Cloud Config Server and VI (Stateless) by avoiding HttpSession.

1.7 When to Use Microservices

This is the most important decision you'll make as an architect. Here's the framework a senior engineer uses:

Signal	Implication
Team > 15-20 developers	Coordination overhead on monolith is killing velocity → Decompose
Need independent scaling	Checkout vs Admin have 100:1 traffic ratio → Microservices
Different release cadences	Payments deploys weekly, UI deploys daily → Decouple
Team < 10 developers	Microservices overhead outweighs benefits → Stay monolith
Domain boundaries unclear	Premature decomposition creates distributed monolith → Wait
No CI/CD, no Docker/K8s	Operational prerequisites missing → Build infra first
Startup / MVP phase	Speed of iteration beats everything → Monolith first
Proven scaling pain points	Specific bottlenecks identified → Strangler Fig Pattern

■ PRACTICE QUESTIONS WITH DETAILED ANSWERS

Every question answered in depth — theory, examples, and real-world context.

Q
1

[Conceptual]

What is a microservice? How does it differ from a module within a monolith?

■ ANSWER:

A **microservice** is an independently deployable unit of software that implements a single business capability, runs in its own process, owns its own database, and communicates with other services exclusively via network APIs (REST, gRPC, or messaging).

A module within a monolith, while it may be logically separated, is compiled and deployed as part of the same artifact. Modules share the same database, the same JVM process, the same deployment pipeline, and often the same team's release schedule. A bug in one module can crash the entire JVM, affecting all modules.

Key differences:

- **Deployment:** Module = deploy everything; Microservice = deploy only that service
- **Database:** Module = shared DB; Microservice = its own DB
- **Process:** Module = same JVM; Microservice = separate JVM/container
- **Failure:** Module failure = potential full crash; Microservice = isolated failure
- **Scaling:** Module = scale everything; Microservice = scale just that service

Example: In an e-commerce app, 'OrderModule' inside a monolith JAR shares database tables with UserModule. 'OrderService' as a microservice has its own database, runs on port 8082, and is deployed independently. Changing the order schema doesn't require testing or redeploying the user functionality.

Q
2

[Conceptual]

Explain the Single Responsibility Principle in the context of microservices with a practical example.

■ **ANSWER:**

SRP in microservices means: **each service has exactly one reason to change**. This reason should be tied to a single business capability or domain.

Practical Example — E-commerce platform:

- **CORRECT:** OrderService only changes when order business rules change (order status transitions, order validation rules, order pricing logic)
- **WRONG:** OrderService also handles user authentication, sends emails, AND manages inventory

Why does violation hurt? If OrderService handles both orders AND emails, and the email provider changes from SendGrid to SES, you must test the entire order processing flow even though nothing order-related changed. Risk increases with unrelated changes.

How to identify SRP violations: Ask 'What would cause this service to change?' If you list more than one unrelated reason, the service has too many responsibilities. 'OrderService changes when: (1) order status rules change OR (2) email template changes OR (3) payment gateway changes' — this service needs to be split.

Q
3

[Conceptual]

What is Loose Coupling? Why is it critical in microservices? Give examples of tight and loose coupling.

■ **ANSWER:**

Loose coupling means that a service can change its internal implementation without forcing changes in the services that depend on it. Services know as little as possible about each other.

TIGHT COUPLING Examples (avoid these):

- Service A imports Service B's Java classes directly (compile-time dependency)
- Service A queries Service B's database table directly (shared database)
- Service A and B share the same deployment pipeline (must deploy together)
- Service A uses Service B's internal data format (internal DTO shared between services)

LOOSE COUPLING Examples (do these):

- Service A calls Service B only via REST API — Service B can change its internal implementation freely
- Service A subscribes to OrderPlaced events from Service B — completely decoupled
- Service A and B can be deployed independently at any time

Real-world impact: At Amazon, when they moved from monolith to microservices, they enforced a rule: 'All services must expose their data through APIs. No direct database access by other teams.' This single rule forced loose coupling across the entire organization.

Q
4

[Conceptual]

What is the 'Database per Service' pattern? Why is it important and what challenges does it create?

■ **ANSWER:**

The **Database per Service** pattern requires that each microservice has its own dedicated data store. No two services share the same database schema, and no service directly accesses another service's database.

Why it's important:

- Schema Independence: OrderService can add/modify/delete tables without breaking UserService
- Technology Choice: OrderService uses MySQL for relational data; ProductService uses MongoDB for flexible catalogs; SessionService uses Redis for fast in-memory access
- Deployment Independence: Database migrations for OrderService don't require UserService downtime
- Failure Isolation: If OrderService's DB goes down, UserService can still log in users

Challenges it creates:

- No ACID transactions across services — if CreateOrder succeeds but DeductInventory fails, you have inconsistent data
- Complex queries that join data across services require API composition or CQRS
- Data duplication — UserService stores userId, OrderService also stores userId+userName for display

Solution to cross-service consistency: Saga Pattern. A saga is a sequence of local transactions. Each step publishes an event. If any step fails, compensating transactions undo previous steps. Example: OrderSaga = CreateOrder → DeductInventory → ProcessPayment → SendConfirmation

Q
5

[Conceptual]

What is Conway's Law? How does it relate to microservices architecture?

■ **ANSWER:**

Conway's Law (Melvin Conway, 1968): 'Organizations which design systems are constrained to produce designs which are copies of the communication structures of these organizations.'

In plain English: your software architecture will mirror your team structure. If you have 4 teams, you'll naturally create 4 major system components. If two teams share an office and communicate daily, their services will be more tightly coupled. If teams are in different countries with 12-hour time differences, their services will be more independently deployable (out of necessity).

Implication for microservices — the 'Inverse Conway Maneuver': Instead of letting team structure determine system design, deliberately structure your teams to match the desired microservices architecture.

- Define service boundaries first (Domain-Driven Design — Bounded Contexts)
- Form teams around those boundaries (Amazon's 'Two Pizza Teams')
- Each team owns one or few services end-to-end (development, deployment, monitoring)

Example: If you want a separate Payment microservice that deploys 5x per day, create a dedicated Payment Team with full ownership. If that team is part of a larger Order Team, Conway's Law predicts the services will be tightly coupled and won't deploy independently.

Q
6

[Scenario]

Your e-commerce monolith becomes extremely slow at checkout during sales. How would you decompose it into microservices? Which service would you extract first and why?

■ **ANSWER:**

This is the classic 'Strangler Fig Pattern' scenario — decompose a monolith incrementally rather than a big-bang rewrite.

Analysis: If checkout is slow during sales, the bottleneck is likely: high volume of order creation, inventory checks, payment processing, and potentially session management.

Decomposition approach — extract services in order of business impact and isolation ease:

- **Step 1 — Extract Session/Auth Service:** Authentication is typically stateless and easy to extract. It has minimal dependencies on other modules. This alone reduces monolith load.
- **Step 2 — Extract Product Catalog Service:** Product browsing is read-heavy and independently scalable. Easy to extract — usually has clean boundaries. Can be cached aggressively with CDN.
- **Step 3 — Extract Inventory Service:** The actual bottleneck during sales. Once extracted, it can be scaled independently to 20+ instances during sales. Use Redis for real-time stock counts.
- **Step 4 — Extract Order Service:** Depends on Inventory being stable. Now order creation is independent.
- **Step 5 — Extract Payment Service:** Usually last because it's the most critical and has compliance requirements (PCI-DSS).

Why NOT extract Payment first? Payment is the most business-critical service. If it breaks during extraction, revenue stops. Always start with lower-risk services. Rule: Extract services from the outside in, starting with the least business-critical.

Q
7

[Scenario]

A startup of 5 developers wants to use microservices. As a senior architect, what would you advise?

■ **ANSWER:**

Advice: **Do NOT start with microservices.** Here's the detailed reasoning:

With 5 developers, the primary constraint is velocity — how fast can you ship features to validate product-market fit. Microservices impose overhead that kills velocity at small scale:

- Infrastructure overhead: You need Docker, Kubernetes, service registry (Eureka), API Gateway, distributed tracing, centralized logging — minimum 3-4 weeks of setup that generates zero user value
- Operational complexity: 5 developers now maintain 10 services, 10 databases, 10 deployment pipelines
- Premature decomposition: Domain boundaries are unclear at MVP stage. Wrong cuts now create a 'distributed monolith' — the worst of both worlds
- Network overhead: Every inter-service call adds latency. With 5 devs, optimizing for latency is premature

RECOMMENDED APPROACH — Modular Monolith:

- Build a well-structured monolith with clear package/module boundaries (UserModule, OrderModule, PaymentModule)
- Enforce strict inter-module contracts (modules communicate via interfaces, not direct class access)
- Use separate database schemas (even within one DB server) per module
- This makes future extraction to microservices much easier when the time comes

When to reconsider: When the team grows to 15+, when specific scaling bottlenecks appear, or when independent deployment becomes a blocker. Then use the Strangler Fig Pattern to extract services one by one.

Q
8

[Conceptual]

List 5 key differences between monolithic and microservices architectures with real-world consequences.

■ ANSWER:

Here are 5 critical differences with production consequences:

- **1. Deployment Unit:** Monolith = one JAR for entire app. Microservices = one JAR per service. Consequence: In a monolith, a CSS change requires deploying the entire backend. One broken test blocks the whole team. In microservices, the UI team deploys 10x/day independently.
- **2. Database Sharing:** Monolith = one shared DB. Microservices = DB per service. Consequence: In a monolith, schema changes require coordination across all teams. A missing index added by one team can degrade everyone's queries. In microservices, teams manage their DB independently.
- **3. Failure Blast Radius:** Monolith = one service crash = everything down. Microservices = isolated failures. Consequence: Amazon's 2013 outage — a monolith would have taken down all services simultaneously. As microservices, only affected services go down.
- **4. Scaling Granularity:** Monolith = scale everything. Microservices = scale specific services. Consequence: During Black Friday, Amazon's checkout service needs 50x capacity. In a monolith, you'd 50x the entire system (all services, all databases). With microservices, only the checkout service and its database scale. Cost reduction: 80-90%.
- **5. Technology Stack:** Monolith = one language/framework. Microservices = polyglot. Consequence: Netflix's machine learning recommendation engine is written in Python (best ML libraries). Their streaming service is Java (best for high-throughput). Their UI is JavaScript. All unified by REST APIs.

Q
9

[Conceptual]

What is a 'Bounded Context' from Domain-Driven Design and how does it map to microservices?

■ **ANSWER:**

A **Bounded Context** (Eric Evans, Domain-Driven Design) is a logical boundary within which a domain model is defined and applicable. The same word can mean different things in different bounded contexts.

Classic Example — 'Customer' means different things:

- In Sales Context: Customer = potential buyer, has CRM notes, lead score, sales rep assigned
- In Order Context: Customer = order placer, has shipping address, order history
- In Support Context: Customer = ticket raiser, has support tier, SLA
- In Billing Context: Customer = account holder, has payment methods, billing address

Mapping to microservices: Each Bounded Context typically becomes one microservice (or a small cluster of services). The Customer in OrderService has only the fields OrderService cares about — NOT the full CRM profile, billing info, support history.

Why this matters: If OrderService needs the customer's payment method, it doesn't store all billing data locally. It calls BillingService API for that specific data. This is Context Mapping — defining how bounded contexts interact.

Practical technique for finding bounded contexts: Event Storming workshops. Get domain experts, developers, and stakeholders in a room. Use sticky notes to map business events (OrderPlaced, PaymentProcessed, ItemShipped). Natural clusters of events reveal bounded contexts, which become your microservices.

Q
10

[Tricky]

Two microservices need to maintain data consistency. You cannot use a distributed transaction (2PC). What are your options?

■ **ANSWER:**

This is one of the most important questions in microservices design. Distributed transactions (2PC — Two-Phase Commit) are avoided because they: require all services to be available simultaneously, cause long-lived locks, reduce availability (CAP theorem), and are complex to implement correctly.

Option 1: Saga Pattern — Choreography

- Each service publishes events after completing its local transaction
- Other services listen for these events and react
- Example: Order Saga — OrderService creates order → publishes OrderCreated event → InventoryService hears it, deducts stock → publishes StockDeducted event → PaymentService hears it, charges card → publishes PaymentCompleted event → OrderService updates status
- Compensating transactions: If PaymentService fails, it publishes PaymentFailed → InventoryService hears it, restores stock → OrderService hears it, cancels order
- Pro: Loosely coupled, no central coordinator. Con: Complex to debug, difficult to track saga state

Option 2: Saga Pattern — Orchestration

- A central Saga Orchestrator (a service or state machine) tells each service what to do
- The orchestrator tracks state and handles failures explicitly
- Pro: Easier to understand, central visibility. Con: Orchestrator becomes a central dependency

Option 3: Eventual Consistency with Idempotency

- Accept that data will be temporarily inconsistent
- Design all operations to be idempotent (can be safely retried)
- Use message queues (Kafka) for guaranteed delivery

Option 4: Outbox Pattern

- Write to local DB AND outbox table in the same local transaction
- A separate process reads the outbox and publishes events
- Guarantees at-least-once delivery with eventual consistency

Q
11

[Conceptual]

Explain the 12-Factor App methodology. Which 3 factors are most critical for Spring Boot microservices?

■ ANSWER:

The 12-Factor App is a set of practices for building modern, cloud-native services. All 12 factors are important, but for Spring Boot microservices specifically, these 3 are most commonly violated and most impactful:

Factor III — Config (Most Violated):

- Problem: Developers hardcode DB passwords, API keys, and service URLs in application.properties and commit to Git. Now production credentials are in version control.
- Solution: All config from environment variables or Spring Cloud Config Server

WRONG:

```
spring.datasource.password=mypassword123
```

RIGHT:

```
spring.datasource.password=${DB_PASSWORD}
```

Factor VI — Stateless Processes (Most Impactful for scaling):

- Problem: Storing user session data in the service's memory. When you add a second instance, users get different sessions on different requests (sticky sessions required).
- Solution: Store sessions in Redis. Every instance can serve any request.

Factor XI — Logs (Most Operationally Critical):

- Problem: Services create rotating log files on disk. In containers, logs are lost when container restarts.
- Solution: Write all logs to stdout. Docker/Kubernetes collects them. ELK Stack (Elasticsearch, Logstash, Kibana) aggregates and makes them searchable.

Q
12

[Conceptual]

What does 'stateless service' mean? Why is it essential for horizontal scaling?

■ ANSWER:

A **stateless service** does not store any information about the client's session between requests. Each request contains all the information needed to process it. The service has no memory of previous interactions with that client.

Stateful vs Stateless:

- **Stateful:** Server stores session data in memory. Request 1: user logs in, server creates session object in HashMap. Request 2: server looks up session in HashMap. If this request goes to Instance B, the HashMap is empty — user is logged out.
- **Stateless:** Client sends JWT token in every request. Server validates the token cryptographically — no stored state needed. Any instance can handle any request.

Why it's essential for horizontal scaling: With stateless services, you can add 10 more instances and a load balancer will distribute traffic equally. All instances are identical and interchangeable. No need for 'sticky sessions' (routing the same user to the same server).

How to make Spring Boot services stateless:

- Replace HttpSession with JWT tokens (stateless authentication)
- Store user preferences in the database, not in-memory
- Use Redis for any cross-request state (shopping cart, rate limiting counters)
- Avoid @SessionScope beans

```
// STATEFUL (avoid):
HttpSession session = request.getSession();
session.setAttribute('cart', cartItems); // Stored in server memory

// STATELESS (correct):
// Cart stored in Redis with userId as key
redisTemplate.opsForValue().set('cart:' + userId, cartItems);
```

Q
13

[Conceptual]

What is the Circuit Breaker pattern? What are the three states of a circuit breaker?

■ ANSWER:

The **Circuit Breaker** pattern prevents cascading failures in a distributed system. Named after the electrical circuit breaker in your home that 'trips' (opens) when it detects dangerous current to protect appliances.

Problem it solves: ServiceA calls ServiceB. ServiceB is slow (taking 30 seconds). Without a circuit breaker, ServiceA's thread pool fills up with requests waiting for ServiceB. Eventually, ServiceA also becomes unresponsive. This is a 'cascading failure' — one slow service brings down the entire chain.

How it works — Three States:

- **CLOSED (Normal):** Requests flow through normally. Circuit breaker counts failures. If failure rate stays below threshold (e.g., < 50%), stays CLOSED.
- **OPEN (Tripped):** Failure rate exceeded threshold. Circuit breaker OPENS immediately — ALL requests fail fast without calling the downstream service. A fallback is returned. Service gets time to recover.
- **HALF-OPEN (Testing):** After a configured wait time (e.g., 30 seconds), circuit breaker allows a few test requests through. If they succeed → transitions to CLOSED. If they fail → transitions back to OPEN.

In Spring Boot with Resilience4j:

```
@CircuitBreaker(name = 'orderService', fallbackMethod = 'getOrderFallback')
public OrderDTO getOrder(Long id) {
    return orderServiceClient.getOrderById(id);
}
```

```
public OrderDTO getOrderFallback(Long id, Exception e) {  
    return new OrderDTO(id, 'UNAVAILABLE'); // Safe default  
}
```

Q
14

[Tricky]

Service A calls Service B which calls Service C. Service C fails. How does this affect A and B? What patterns prevent cascading failure?

■ **ANSWER:**

Without protection, this is a **cascading failure scenario**:

- Service C fails → its thread pool is full of timed-out requests
- Service B's calls to C all block for timeout duration (say 30 seconds)
- Service B's thread pool fills with threads waiting for C
- Service A's calls to B now also block, waiting for B to respond
- Service A's thread pool fills → A becomes unresponsive
- Total system failure from ONE failing service

Patterns to prevent cascading failure:

- **Circuit Breaker**: At Service B's call to C. When C starts failing, the circuit opens. B immediately returns a fallback. B's threads are freed. A gets fast responses (fallback).
- **Timeouts**: B's call to C has a 2-second timeout. After 2 seconds, B stops waiting and returns a fallback. Threads are freed immediately.
- **Bulkhead Pattern**: B allocates a separate, limited thread pool for C calls. Even if all those threads are consumed, the rest of B's thread pool serves other operations.
- **Fallback**: B returns cached data or a safe default when C fails, allowing A to function with degraded but acceptable data.

Implementation rule: Every network call should have: timeout + circuit breaker + fallback. This is the holy trinity of resilient microservices.

Q
15

[Conceptual]

What is a 'Distributed Monolith' and why is it considered worse than a true monolith?

■ **ANSWER:**

A **Distributed Monolith** is a system that is deployed as multiple separate services but is tightly coupled — it has all the disadvantages of both monoliths and microservices, and the benefits of neither.

Signs you have a distributed monolith:

- Services share a database — changing one service's schema breaks others
- Services must be deployed together — deploying OrderService requires deploying UserService too
- Services make synchronous calls in long chains — A calls B calls C calls D
- Services import and use each other's internal Java classes (shared libraries)
- A single service going down takes down many others (no fault isolation)

Why it's WORSE than a monolith:

- All the operational complexity of microservices (50 deployments, 50 logs, network latency)

- None of the benefits (can't scale independently, can't deploy independently, failures cascade)
- Debugging is harder — a bug manifests across multiple services and logs

Root cause: Services were split along technical lines (UI Service, Business Logic Service, Data Service) instead of domain lines. Rule: Split along business capabilities (Order Service, User Service, Payment Service) guided by Domain-Driven Design.

Q
16

[Design]

Design a microservices architecture for an online food delivery platform. List the services, their responsibilities, and their databases.

■ **ANSWER:**

Here's a production-grade microservices design for a Swiggy/Zomato-like platform:

- **User Service:** Registration, login, profile, addresses. DB: PostgreSQL (relational user data + Redis for sessions/JWT blacklist)
- **Restaurant Service:** Restaurant profiles, menus, working hours, locations. DB: MongoDB (flexible menu structure with varying attributes per dish)
- **Order Service:** Order creation, order status management, order history. DB: PostgreSQL (ACID properties critical for order state transitions)
- **Inventory/Menu Availability Service:** Real-time item availability, stock counts. DB: Redis (in-memory for sub-millisecond reads during high traffic)
- **Payment Service:** Payment processing, refunds, payment history. DB: PostgreSQL (financial data needs ACID, PCI-DSS compliance)
- **Delivery/Logistics Service:** Delivery partner assignment, live tracking, route optimization. DB: PostGIS (geospatial queries for nearest delivery partner)
- **Notification Service:** SMS, email, push notifications. DB: MongoDB (flexible notification templates) + Kafka consumer
- **Search Service:** Restaurant/dish search, filters, ranking. DB: Elasticsearch (full-text search, geo-search)
- **Rating & Review Service:** Reviews for restaurants and delivery partners. DB: MongoDB (schemaless reviews)
- **Pricing/Offer Service:** Promo codes, surge pricing, offers. DB: Redis (fast lookup during order placement)

Communication: User Service → API Gateway → microservices. Async events via Kafka: OrderPlaced → [Inventory, Notification, Delivery]. Sync REST calls: Order → Payment, Order → Inventory.

Infrastructure: Eureka for discovery, Spring Cloud Gateway at edge, separate Kubernetes namespace per service, ELK for logs, Jaeger for distributed tracing.

Q
17

[Conceptual]

What is polyglot persistence? Give an example of when different databases make sense.

■ **ANSWER:**

Polyglot persistence means using different database technologies for different services based on the specific data access patterns and requirements of each service, rather than using one database for everything.

Why it matters: No single database is optimal for all workloads. Using the right DB for each service's access pattern dramatically improves performance and developer productivity.

Real examples from the food delivery platform:

- **User Service** → **PostgreSQL**: User data is relational (users have addresses, have orders, have payment methods). ACID transactions needed. PostgreSQL excels here.
- **Menu Service** → **MongoDB**: Restaurant menus vary wildly. Pizza has toppings, sizes, crusts. Sushi has fish types, rolls, sides. Schemaless MongoDB handles variable attributes naturally without endless NULL columns.
- **Search Service** → **Elasticsearch**: 'Find restaurants near me serving biryani under 200 rupees with 4+ stars and available now.' This complex query requires full-text search + geo-search + aggregations — Elasticsearch is built for this.
- **Session/Rate Limiting** → **Redis**: Sub-millisecond reads, TTL-based expiry, atomic operations. Perfect for sessions, API rate limiting, real-time counters.
- **Delivery Tracking** → **Apache Cassandra**: High write throughput (GPS updates every 5 seconds from thousands of drivers), time-series queries. Cassandra's wide-column model handles this efficiently.

Downside: Your team must know multiple database technologies. Trade-off between performance and operational complexity.

Q
18

[Conceptual]

What is the Saga Pattern? Explain choreography vs orchestration.

■ **ANSWER:**

The **Saga Pattern** is a way to manage data consistency across microservices without distributed transactions (2PC). A saga is a sequence of local transactions, where each transaction publishes events that trigger the next transaction. If any step fails, compensating transactions undo the previous steps.

Approach 1: Choreography — Event-driven, no central coordinator

- Each service publishes events and listens for events from other services
- OrderService creates order → publishes OrderCreated → InventoryService deducts stock → publishes StockDeducted → PaymentService charges → publishes PaymentSucceeded → OrderService marks CONFIRMED
- On failure: PaymentService fails → publishes PaymentFailed → InventoryService listens → restores stock → publishes StockRestored → OrderService listens → marks CANCELLED
- Pro: Loose coupling, no central point of failure. Con: Hard to track saga state, debugging is complex

Approach 2: Orchestration — Central saga coordinator

- A dedicated OrderSagaOrchestrator service coordinates the saga
- Orchestrator tells InventoryService 'deduct stock for order 123', waits for response, then tells PaymentService 'charge for order 123', etc.
- Orchestrator tracks state: [INVENTORY_DEDUCTED, PAYMENT_PENDING, ...]
- Pro: Easy to visualize and debug, clear state tracking. Con: Orchestrator is a coupling point

When to use which: Choreography for simple flows with 2-3 steps. Orchestration for complex, long-running sagas with many steps, or when visibility into saga state is important.

Q
19

[Conceptual]

What is service mesh? How does it differ from an API gateway?

■ **ANSWER:**

A **Service Mesh** is an infrastructure layer that handles service-to-service communication within a microservices ecosystem. Popular implementations: Istio, Linkerd, Consul Connect.

How it works: Each service gets a 'sidecar proxy' (usually Envoy) injected alongside it. ALL network traffic goes through this proxy. The control plane (Istio) configures all proxies centrally.

Service Mesh provides:

- mTLS (mutual TLS) encryption between all services automatically
- Distributed tracing without changing application code
- Circuit breaking, retries, timeouts at the network level
- Traffic splitting (canary deployments — 10% to new version)
- Observability — metrics for every service-to-service call

API Gateway vs Service Mesh — key difference:

- **API Gateway:** North-South traffic (external clients → internal services). Single entry point. User-facing concerns (auth, rate limiting, routing).
- **Service Mesh:** East-West traffic (service to service within the cluster). Handles internal communication. Infrastructure concerns (encryption, tracing, retries).

They complement each other. API Gateway handles what clients send in. Service Mesh handles how services talk to each other internally. In a Kubernetes environment: API Gateway (Spring Cloud Gateway or Nginx) + Istio service mesh together give complete control.

Q
20

[Design]

How would you handle cross-cutting concerns like logging, authentication, and rate limiting in a microservices ecosystem?

■ **ANSWER:**

Cross-cutting concerns are functionalities that apply across multiple services. In a microservices ecosystem, they should be centralized — not duplicated in every service.

Authentication & Authorization:

- Handle at the API Gateway using JWT validation (JWTs are stateless — any service can validate them with the public key)
- Gateway validates JWT, extracts userId, adds X-User-Id header to downstream requests
- Individual services trust X-User-Id header (they're behind the gateway, not directly exposed)
- Fine-grained authorization (can user X access resource Y?) done at the individual service

Logging:

- Each service writes structured JSON logs to stdout
- Gateway adds X-Request-Id header — propagated to all downstream services via Feign interceptor
- All services include X-Request-Id in their logs — allows tracing a single user request across 10 services

- ELK Stack (Elasticsearch + Logstash + Kibana) aggregates all logs. Search by request ID to see the full journey.

Rate Limiting:

- Coarse-grained rate limiting at the API Gateway (e.g., 100 req/min per IP)
- Fine-grained rate limiting at individual services for sensitive operations (e.g., 3 payment attempts/hour per user)

Distributed Tracing:

- Use Micrometer + Zipkin/Jaeger. Automatically instruments Feign calls.
 - Trace ID propagated across all service calls — visualize the complete request chain in Jaeger UI
-

Service Registry & Discovery — Eureka Server

How microservices find each other dynamically in a cloud world where IPs constantly change.

2.1 The Problem: Why Service Discovery?

In a traditional application, if Service A needs to call Service B, you configure a static IP and port: `http://192.168.1.100:8082`. This breaks immediately in a cloud environment because:

- **Dynamic IPs:** Kubernetes/Docker assigns new IPs on every container restart. Your static configuration is stale within minutes.
- **Multiple Instances:** Service B runs on 5 machines: `:8082`, `:8083`, `:8084`, `:8085`, `:8086`. Which one does Service A call? How does it distribute load?
- **Auto-scaling:** During a sale, Service B scales from 5 → 20 instances. Service A must know about the 15 new instances automatically.
- **Instance Failures:** Instance 3 of Service B crashes. Service A must stop sending requests to it immediately. Static config can't react to this.

Service Discovery solves this by maintaining a real-time registry of all healthy service instances. Services register themselves, and consumers look up service instances by name (not by IP).

■ ANALOGY

Service Discovery is like your phone's contact book. Instead of remembering your friend's phone number (which changes when they change SIM cards), you save them as 'Rahul' and the system looks up the current number. Services register their current IP:port under a name (e.g., 'order-service'), and other services look them up by name.

2.2 Two Patterns of Service Discovery

Pattern	How It Works	Examples
Client-Side Discovery	Client (Service A) queries the registry, gets list of instances, picks one, calls it directly	Eureka + Spring Cloud LB, Netflix Ribbon
Server-Side Discovery	Client calls a router/LB. Router queries registry and forwards to chosen instance	AWS ALB + ECS, Kubernetes Service

Eureka uses **Client-Side Discovery**. Each client (microservice) fetches the full registry from Eureka, caches it locally, and chooses an instance using Spring Cloud LoadBalancer (Round Robin by default).

2.3 How Eureka Works — Complete Internal Mechanics

Eureka Server is a REST-based service registry. Here's the complete lifecycle:

2.3.1 Registration — Service Startup

When a Eureka client (microservice) starts up, it sends a POST request to Eureka Server:

```
// ■■ Registration Request to Eureka ■■
POST /eureka/apps/ORDER-SERVICE
Body: {
  'instance': {
    'hostname': '10.0.1.45',
    'app': 'ORDER-SERVICE',
    'ipAddr': '10.0.1.45',
    'port': { '$': 8082, '@enabled': 'true' },
    'status': 'UP',
    'healthCheckUrl': 'http://10.0.1.45:8082/actuator/health',
    'statusPageUrl': 'http://10.0.1.45:8082/actuator/info',
    'metadata': { 'management.port': '8082' }
  }
}
Eureka stores this in memory:
Registry: { ORDER-SERVICE: [ { host:10.0.1.45, port:8082, status:UP } ] }
```

2.3.2 Heartbeat — Keeping the Registration Alive

After registration, every **30 seconds** (configurable), the client sends a heartbeat:

```
// ■■ Heartbeat Mechanism ■■
PUT /eureka/apps/ORDER-SERVICE/10.0.1.45:order-service:8082
Response: 200 OK (Eureka acknowledges the heartbeat)
# If 3 consecutive heartbeats are missed (90 seconds):
# Eureka marks the instance as DOWN, then eventually removes it
# Configurable in client:
eureka.instance.lease-renewal-interval-in-seconds: 30 # heartbeat interval
eureka.instance.lease-expiration-duration-in-seconds: 90 # timeout before eviction
```

2.3.3 Discovery — Fetching the Registry

Service consumers fetch the registry to discover other services:

```
// ■■ Registry Fetch ■■
# Initial full fetch:
GET /eureka/apps
Response: Full registry XML/JSON with ALL registered services
# Subsequent incremental deltas (every 30 seconds):
GET /eureka/apps/delta
Response: Only changes since last fetch (more efficient)
# Client caches the registry locally.
# Cache refresh: every 30 seconds (default).
# This means there's up to 30-second delay before a new instance is visible.
```

2.3.4 Self-Preservation Mode — CRITICAL Concept

Self-Preservation Mode is Eureka's protection against incorrectly deregistering healthy instances during a network partition.

The problem it solves: Imagine Eureka Server and 50 microservices are in different network segments. A network partition occurs — Eureka can't receive heartbeats from any service for 5 minutes. Without self-preservation, Eureka would deregister ALL 50 services. But the services are actually healthy! They're just not reachable from Eureka due to the network issue. If Eureka removes them, no service can call any other.

Self-Preservation logic:

- Eureka calculates the expected heartbeat rate: $\text{number of services} \times (60/\text{renewal-interval})$
- If actual received heartbeats fall below 85% of expected rate in the last 15 minutes...
- Eureka enters Self-Preservation Mode: STOPS evicting instances from the registry
- Dashboard shows the red warning: 'EMERGENCY! EUREKA MAY BE INCORRECTLY CLAIMING INSTANCES ARE UP'
- This is CORRECT behavior in production — better to serve stale data than to mass-deregister healthy services

IMPORTANT

In DEVELOPMENT, disable self-preservation to avoid seeing phantom services. In PRODUCTION, ALWAYS keep it enabled. It protects against network blips causing mass deregistration.

2.4 Complete Eureka Server Setup

2.4.1 Project Structure

```
// ■■ Eureka Server Project Structure ■■
eureka-server/
■■■ pom.xml
■■■ src/main/
■■■ java/com/example/eureka/
■ ■■■ EurekaServerApplication.java
■■■ resources/
■■■ application.yml
```

2.4.2 pom.xml

```
// ■■ pom.xml – Eureka Server ■■
<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>3.2.0</version>
</parent>
<properties>
<spring-cloud.version>2023.0.0</spring-cloud.version>
</properties>
<dependencies>
<dependency>
```

```

<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-netflix-eureka-server</artifactId>
</dependency>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
</dependencies>
<dependencyManagement>
<dependencies>
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-dependencies</artifactId>
<version>${spring-cloud.version}</version>
<type>pom</type>
<scope>import</scope>
</dependency>
</dependencies>
</dependencyManagement>

```

2.4.3 Main Application Class

```

// ■■ EurekaServerApplication.java ■■
package com.example.eureka;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.netflix.eureka.server.EnableEurekaServer;

@SpringBootApplication
@EnableEurekaServer // This annotation activates the Eureka Server
public class EurekaServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(EurekaServerApplication.class, args);
    }
}

// That's it! @EnableEurekaServer does everything.
// It activates the registry, the REST endpoints, the dashboard,
// and all Eureka server logic via auto-configuration.

```

2.4.4 application.yml — Complete Configuration with Comments

```

// ■■ application.yml – Full Eureka Server Configuration ■■
server:
port: 8761 # Standard Eureka port. Industry convention, not mandatory.
spring:
application:
name: eureka-server
eureka:
client:
register-with-eureka: false
# The Eureka Server should NOT register itself in the registry.
# It IS the registry. Setting true would cause it to look for
# another Eureka server to register with (needed in HA clusters).

```

```

fetch-registry: false
# The Eureka Server does not need to fetch the registry from
# itself. It IS the source of truth.
service-url:
defaultZone: http://localhost:8761/eureka/
# URL where this Eureka server's REST API is hosted.
# In HA mode, this points to peer servers.
server:
enable-self-preservation: false
# FALSE for development only. Prevents stale instances from
# staying in registry when you stop/restart services frequently.
# In PRODUCTION: set to true (default).
eviction-interval-timer-in-ms: 5000
# How often Eureka checks for expired instances.
# Default 60000ms (1 min). Reduce in DEV for faster cleanup.
renewal-percent-threshold: 0.85
# 85% of expected heartbeats must be received to stay out
# of self-preservation mode.
instance:
hostname: localhost
management:
endpoints:
web:
exposure:
include: health,info,metrics

```

2.5 Eureka Client — Complete Configuration

```

// ■■ application.yml – Eureka Client (Microservice) ■■
# application.yml for any microservice (e.g., order-service)
server:
port: 8082
spring:
application:
name: order-service
# CRITICAL: This is the service ID used for discovery.
# Other services discover this by calling lb://order-service.
# Must match exactly (case-insensitive in Eureka, but keep consistent).
eureka:
client:
register-with-eureka: true # Default. Register this service.
fetch-registry: true # Default. Fetch other services' registry.
service-url:
defaultZone: http://localhost:8761/eureka/
# In production: http://eureka-1:8761/eureka/,http://eureka-2:8762/eureka/
# Comma-separated list for HA.
registry-fetch-interval-seconds: 30
# How often to refresh the local cache of the registry.
# Reduce to 5 in DEV for faster discovery.
instance:

```

```

prefer-ip-address: true
# Register using IP address instead of hostname.
# Essential in Docker/K8s where hostnames aren't resolvable.
instance-id: ${spring.application.name}:${server.port}
# Unique ID for this instance in Eureka.
# Format: order-service:8082
# Without this, multiple instances may overwrite each other's registration.
lease-renewal-interval-in-seconds: 30
# Heartbeat interval. Every 30 seconds.
lease-expiration-duration-in-seconds: 90
# If Eureka doesn't hear for 90 seconds, mark as DOWN.
health-check-url-path: /actuator/health
status-page-url-path: /actuator/info

```

2.6 Programmatic Discovery with DiscoveryClient

```

// ■■ Using DiscoveryClient Programmatically ■■
import org.springframework.cloud.client.ServiceInstance;
import org.springframework.cloud.client.discovery.DiscoveryClient;
@RestController
public class DiscoveryController {
    @Autowired
    private DiscoveryClient discoveryClient;
    @GetMapping('/services')
    public List<String> getAllServices() {
        // Returns: ['order-service', 'user-service', 'payment-service']
        return discoveryClient.getServices();
    }
    @GetMapping('/instances/{serviceName}')
    public List<ServiceInstance> getInstances(@PathVariable String serviceName) {
        // Returns all instances of 'order-service' with their IPs/ports
        return discoveryClient.getInstances(serviceName);
    }
    @GetMapping('/call-order/{id}')
    public String callOrderService(@PathVariable Long id) {
        List<ServiceInstance> instances = discoveryClient.getInstances('order-service');
        if (instances.isEmpty()) {
            throw new RuntimeException('order-service not available');
        }
        // Pick first instance (in production, use load balancer)
        ServiceInstance instance = instances.get(0);
        String url = instance.getUri() + '/api/orders/' + id;
        return restTemplate.getForObject(url, String.class);
    }
}

```

2.7 High Availability — Eureka Cluster (Production Setup)

```

// ■■ Eureka HA Cluster Configuration ■■

```

```
# Peer 1: application-peer1.yml
server.port: 8761
spring.application.name: eureka-server
eureka.instance.hostname: eureka-peer1
eureka.client.register-with-eureka: true # Register WITH peer2
eureka.client.fetch-registry: true # Fetch from peer2
eureka.client.service-url.defaultZone: http://eureka-peer2:8762/eureka/

# Peer 2: application-peer2.yml
server.port: 8762
spring.application.name: eureka-server
eureka.instance.hostname: eureka-peer2
eureka.client.register-with-eureka: true # Register WITH peer1
eureka.client.fetch-registry: true # Fetch from peer1
eureka.client.service-url.defaultZone: http://eureka-peer1:8761/eureka/

# Microservice clients register with BOTH peers:
eureka.client.service-url.defaultZone:
http://peer1:8761/eureka/,http://peer2:8762/eureka/

# Now if peer1 goes down:
# - All new registrations go to peer2
# - Peer2 already has full registry (replicated from peer1)
# - Clients fall back to peer2 automatically
# - System remains operational
```

★ SENIOR

In production at large companies, Eureka is deployed across 3+ Availability Zones. Each zone has its own Eureka cluster. Services register with all zones. Even if an entire data center goes down, service discovery continues. Netflix runs Eureka on AWS with regional + zonal replication.

■ PRACTICE QUESTIONS WITH DETAILED ANSWERS

Every question answered in depth — theory, examples, and real-world context.

Q
1

[Conceptual]

What problem does Service Discovery solve? Why can't we use static IP addresses?

■ ANSWER:

Service Discovery solves the problem of dynamically locating service instances in a cloud environment where static configuration is impossible.

Why static IPs fail in modern infrastructure:

- **Container Restarts:** Every time a Docker container or Kubernetes pod restarts, it gets a new IP address. If Service A has hardcoded `http://10.0.1.45:8082` for Service B, and Service B's container restarts on IP `10.0.1.67`, all calls from A fail immediately.
- **Auto-scaling:** If Order Service scales from 2 to 10 instances during a sale, you'd need to update every other service's configuration file and restart them — during peak traffic.
- **Multi-environment:** Dev, staging, and production have different IPs. Managing different config files for each environment while keeping them in sync is error-prone.

- **Zero-downtime Deployments:** Rolling deployments spin up new instances with new IPs while shutting down old ones. Static configs can't track this in real time.

Service Discovery solution: Services register their current IP:port at startup. Consumers look up services by name. Discovery happens dynamically at runtime, always returning the current, healthy instances.

Q
2

[Conceptual]

What is the Eureka heartbeat mechanism? What happens when heartbeats stop?

■ **ANSWER:**

The heartbeat mechanism is how Eureka keeps its registry accurate and removes dead instances. It's a periodic 'I'm alive' signal from each registered service to Eureka Server.

Complete heartbeat flow:

- **After Registration:** Every 30 seconds (lease-renewal-interval-in-seconds), service sends: `PUT /eureka/apps/ORDER-SERVICE/{instanceId}`
- **Eureka Response:** 200 OK — heartbeat acknowledged. Eureka resets the expiration timer.
- **Missed Heartbeat:** If Eureka doesn't receive a heartbeat within 90 seconds (lease-expiration-duration-in-seconds = 3 missed heartbeats), it marks the instance as DOWN.
- **Eviction:** After being marked DOWN, Eureka removes the instance from the registry (unless in self-preservation mode).

Important: The heartbeat is sent by the CLIENT, not polled by the server. This is a 'push' model. Compare to active health checks (where the server polls each service's /health endpoint) — Eureka uses heartbeats for efficiency.

```
# Heartbeat configuration:
eureka.instance.lease-renewal-interval-in-seconds: 30 # Send every 30 seconds
eureka.instance.lease-expiration-duration-in-seconds: 90 # Evict after 90 seconds
```

Q
3

[Conceptual]

What is Eureka Self-Preservation Mode? Why does it exist?

■ **ANSWER:**

Self-Preservation Mode is a safety mechanism in Eureka Server that prevents mass deregistration of instances during temporary network issues.

The problem without self-preservation: Imagine 100 microservice instances are running and healthy. A network switch fails between them and the Eureka server. Eureka stops receiving heartbeats from all 100 instances. Without self-preservation, after 90 seconds, Eureka evicts ALL 100 instances. The network switch comes back up — but now no service can discover any other service. Complete service discovery outage for a healthy system.

Self-preservation logic:

- Eureka expects N heartbeats per minute (N = registered instances × renewals-per-minute)
- If actual received heartbeats drop below 85% of expected (threshold configurable)
- Eureka enters self-preservation: STOPS evicting instances, even if they've missed heartbeats

- This preserves the last known-good state of the registry

Trade-off: During self-preservation, dead instances may remain in the registry. Callers might get routed to dead services and get connection refused errors. This is handled by retry logic and circuit breakers in the calling services.

Configuration:

```
# Production (default - keep enabled):
eureka.server.enable-self-preservation: true

# Development (disable for clean testing):
eureka.server.enable-self-preservation: false
eureka.server.eviction-interval-timer-in-ms: 2000
```

Q
4

[Conceptual]

What is the significance of `spring.application.name` in a Eureka client?

■ **ANSWER:**

`spring.application.name` is one of the most important properties in a microservice. It serves as the **service identifier** — the name under which this service registers in Eureka and by which other services discover it.

What it controls:

- **Eureka Registration Key:** The service registers as ORDER-SERVICE (uppercase) in Eureka registry. This is the key in the registry map.
- **Service Discovery Name:** Other services call `lb://order-service` to discover and call it. The 'order-service' here must match `spring.application.name` (case-insensitive).
- **Load Balancing Target:** Feign `@FeignClient(name='order-service')` and Gateway uri: `lb://order-service` both use this name.
- **Actuator Info:** Appears in `/actuator/info` and health checks.
- **Distributed Tracing:** Appears as service name in Zipkin/Jaeger traces.

```
# order-service's application.yml:
spring:
  application:
    name: order-service # Register as 'ORDER-SERVICE' in Eureka

# In user-service, calling order-service:
@FeignClient(name = 'order-service') # MUST match the name above
```

Critical mistake: Two different services using the same `spring.application.name`. They'll register under the same key. Eureka will think they're different instances of the same service. Load balancer will route order calls to user-service instances — catastrophic confusion.

Q
5

[Code]

Write the minimum configuration needed to set up a Eureka Server on port 8761.

■ **ANSWER:**

Here is the complete minimum setup — 3 files needed:

1. pom.xml dependency:

```
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-netflix-eureka-server</artifactId>
```

```
</dependency>
```

2. Main Application class:

```
@SpringBootApplication
@EnableEurekaServer
public class EurekaServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(EurekaServerApplication.class, args);
    }
}
```

3. application.yml:

```
server:
port: 8761
spring:
application:
name: eureka-server
eureka:
client:
register-with-eureka: false
fetch-registry: false
```

That's all you need. Spring Boot auto-configuration handles the rest: the dashboard, REST endpoints, registry, heartbeat processing, and eviction logic.

Q
6

[Tricky]

You have Service A calling Service B. Eureka still shows an old instance of B that has crashed. What happens? How do you mitigate?

■ ANSWER:

This is a real production scenario called the 'stale registry' problem.

What happens:

- Service B's instance crashed 45 seconds ago. Eureka hasn't evicted it yet (90-second timeout).
- Service A fetches registry — old instance IP still there (registry cache refresh: 30 seconds).
- Spring Cloud LB picks the dead instance via Round Robin.
- Service A sends request to dead IP — gets Connection Refused or timeout.
- From user's perspective: request fails or is very slow.

Total worst-case stale window: 90s (eviction) + 30s (LB cache) = 120 seconds of potentially routing to dead instance.

Mitigation strategies:

- **Retry:** Configure Feign/WebClient to retry on connection failure, on a DIFFERENT instance.
- **Circuit Breaker:** After N failures to an instance, circuit opens. Fast-fail instead of timing out.
- **Reduce timeouts:** Set connect-timeout to 1-2 seconds. Fail fast, retry on next instance.
- **Health check-based routing:** Spring Cloud LB can filter instances based on /actuator/health before routing.


```
# Feign retry configuration:
feign:
  client:
  config:
  order-service:
  connect-timeout: 1000 # Fail fast on dead instance
  read-timeout: 3000
```

Q
7

[Scenario]

You start 3 instances of Order Service. They all register with Eureka. How does Service A know which instance to call?

■ **ANSWER:**

Here's the complete flow:

- **Step 1 — Registration:** All 3 instances start and register with Eureka: ORDER-SERVICE → [{host1:8082}, {host2:8082}, {host3:8082}]
- **Step 2 — Discovery:** Service A fetches the registry. Its local cache now has: ORDER-SERVICE → [host1, host2, host3]
- **Step 3 — Load Balancing:** Spring Cloud LoadBalancer applies Round Robin: Request 1 → host1, Request 2 → host2, Request 3 → host3, Request 4 → host1...

How this is triggered in Feign:

```
@FeignClient(name = 'order-service')
public interface OrderClient {
  @GetMapping('/api/orders/{id}')
  Order getOrder(@PathVariable Long id);
}
// Feign sees 'order-service' → asks Spring Cloud LB
// LB picks one instance → Feign calls that instance's URL
```

How this is triggered in Gateway:

```
uri: lb://order-service # 'lb://' tells Gateway to use Spring Cloud LB
# Gateway asks LB, LB picks one instance, Gateway forwards request
```

Q
8

[Conceptual]

What happens if the Eureka Server itself goes down? Can microservices still communicate?

■ **ANSWER:**

YES — microservices CAN still communicate after Eureka goes down, for a limited time. This is by design. Here's why:

Client-side caching: Each microservice client caches the registry locally. The cache is populated when Eureka was last available. When Eureka goes down, services continue using their cached registry.

What happens step by step:

- **Eureka goes down at T=0**
- **T=0 to T=30s:** All services use their cached registry. Everything works normally.
- **T=30s:** Clients try to refresh registry — Eureka is down, refresh fails. They continue with stale cache.
- **T=30s to T=∞:** Services continue working with stale cache. New services can't be discovered. Crashed services aren't removed from cache.

- **Risk:** If a service instance crashes while Eureka is down, its IP stays in the cache. Calls to it will fail (mitigated by circuit breakers).

This is the beauty of client-side discovery — the registry is decentralized into each client's memory. Eureka going down doesn't immediately stop service communication.

In production: Use Eureka HA (cluster of 3 nodes across 3 AZs). Probability of all 3 going down simultaneously is extremely low.

Q
9

[Tricky]

What does 'EMERGENCY! EUREKA MAY BE INCORRECTLY CLAIMING INSTANCES ARE UP' mean on the dashboard?

■ **ANSWER:**

This warning means Eureka has entered **Self-Preservation Mode**.

Root cause: Eureka calculated that it's receiving fewer heartbeats than expected (below 85% threshold). It suspects a network issue and stopped evicting instances.

When you see this:

- **In Development:** You likely started/stopped services frequently, or your Eureka server and services were on different network segments, or you set very aggressive eviction timers. Solution: Set `enable-self-preservation: false` in dev.
- **In Production:** Something is wrong — network partition, high load causing heartbeat delays, or many services dying simultaneously. Investigate the root cause. Do NOT disable self-preservation in production.

What it means for your system: Eureka will keep showing instances as UP even if they might be down. Your circuit breakers and retry logic must handle the resulting connection failures gracefully.

Common dev mistake: Seeing this warning and panicking. In development with self-preservation disabled, you'll never see it. In production, it's a signal to investigate network health, not a disaster by itself.

Q
10

[Design]

Design a Highly Available Eureka setup for a production system.

■ **ANSWER:**

A production-grade Eureka HA setup typically uses 3 Eureka server nodes across 3 Availability Zones:

```
# Network Layout:
# AZ-1: Eureka-1 (10.0.1.10:8761)
# AZ-2: Eureka-2 (10.0.2.10:8762)
# AZ-3: Eureka-3 (10.0.3.10:8763)

# Each Eureka registers with the other two:
# eureka-1: defaultZone = http://eureka-2:8762/eureka/,http://eureka-3:8763/eureka/
# eureka-2: defaultZone = http://eureka-1:8761/eureka/,http://eureka-3:8763/eureka/
# eureka-3: defaultZone = http://eureka-1:8761/eureka/,http://eureka-2:8762/eureka/

# Microservices register with ALL THREE:
eureka.client.service-url.defaultZone:
http://eureka-1:8761/eureka/,http://eureka-2:8762/eureka/,http://eureka-3:8763/eureka/
```

Resilience analysis:

- **1 node down:** 2 nodes serve discovery. Remaining 2 replicate to each other. No impact.
- **2 nodes down:** 1 node serves discovery from its registry. Clients use cache. Degraded but operational.
- **All 3 down:** Clients use local cache. No new discovery. In a stable system, this is acceptable for minutes.

Additional production considerations:

- Use AWS Route53 with health checks for Eureka DNS — fail over automatically
 - Put Eureka servers behind an internal load balancer
 - Monitor Eureka metrics via Spring Boot Actuator + Prometheus
 - Alert on self-preservation mode activation (custom actuator health indicator)
-

Feign Client — Inter-Service Communication

The elegant, declarative way microservices talk to each other — zero boilerplate.

3.1 The Problem with RestTemplate

Before Feign, inter-service calls used RestTemplate. Here's the pain:

```
// ■■ RestTemplate – The Verbose Old Way ■■
// OLD WAY – RestTemplate for every service call
@Service
public class UserService {
    @Autowired
    private RestTemplate restTemplate;

    public List<Order> getUserOrders(Long userId) {
        // Manual URL construction – error-prone
        String url = 'http://order-service/api/orders?userId=' + userId;
        // Manual response handling
        ResponseEntity<List<Order>> response =
            restTemplate.exchange(url, HttpMethod.GET, null,
                new ParameterizedTypeReference<List<Order>>() {});
        // Manual status code checking
        if (response.getStatusCode() == HttpStatus.OK) {
            return response.getBody();
        } else if (response.getStatusCode() == HttpStatus.NOT_FOUND) {
            throw new NotFoundException('Orders not found for user: ' + userId);
        }
        throw new RuntimeException('Unexpected status: ' + response.getStatusCode());
    }
}

// Problems:
// 1. String concatenation for URLs – typos cause runtime failures
// 2. Manual exception mapping
// 3. Boilerplate ResponseEntity handling
// 4. No load balancing without @LoadBalanced + extra config
// 5. Multiply this by 20 service calls = unmaintainable code
```

3.2 What Is Feign? Internal Mechanics

Feign is a **declarative HTTP client** from Netflix. You write a Java interface describing what you want to call. Feign generates the implementation at runtime using Java Dynamic Proxies. You just write the interface, Feign writes the code.

Internal mechanics:

- **@EnableFeignClients**: Scans classpath for @FeignClient interfaces. Creates a Spring bean for each.
- **Dynamic Proxy**: For each @FeignClient interface, Feign creates a JDK dynamic proxy that implements your interface. When you call orderClient.getOrder(123), the proxy intercepts it.
- **Contract**: The proxy reads @GetMapping, @RequestParam, @PathVariable annotations and constructs the HTTP request.
- **Load Balancing**: The service name ('order-service') is resolved via Spring Cloud LoadBalancer, which queries Eureka for instances.
- **HTTP Call**: Feign makes the actual HTTP call using its underlying HTTP client (OkHttp or HttpClient).
- **Deserialization**: The JSON response is deserialized into the return type using Jackson (or configured ObjectMapper).

■ ANALOGY

Feign is like hiring a personal assistant. You give them a job description: 'Whenever I ask for orders, call the order department at extension order-service, request JSON format, and bring it back as a list.' You don't care HOW they make the call, route it, or handle the response. You just say 'get me the orders for user 123' and the assistant does everything else.

3.3 Complete Feign Setup

3.3.1 Dependencies (Spring Boot 3.x)

```
// ■■ pom.xml ■■
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-openfeign</artifactId>
<!-- Includes: Feign core, Spring integration, Jackson encoder/decoder -->
</dependency>
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
<!-- Required for service discovery by name -->
</dependency>
<!-- For circuit breaker support with Feign: -->
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-circuitbreaker-resilience4j</artifactId>
</dependency>
```

3.3.2 Enable Feign in Main Class

```
// ■■ Main Application ■■
@SpringBootApplication
@EnableFeignClients
// Scans all @FeignClient interfaces in the current package and sub-packages.
// To limit scan: @EnableFeignClients(basePackages = 'com.example.clients')
```

```
// To specify explicitly: @EnableFeignClients(clients = {OrderServiceClient.class})
public class UserServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(UserServiceApplication.class, args);
    }
}
```

3.4 Complete Feign Client Interface

```
// ■■ OrderServiceClient.java – Complete Feign Interface ■■
// DTOs
public record OrderDTO(Long id, String product, Double amount, String status) {}
public record CreateOrderRequest(Long userId, String product, Integer qty) {}

// The Feign Client Interface
@FeignClient(
    name = 'order-service',
    // 'name' = Eureka service ID (spring.application.name of order-service)
    // Spring Cloud LB resolves this to an actual IP:port via Eureka
    path = '/api/orders',
    // Optional base path. All methods append to this.
    // getOrderById -> GET /api/orders/{id}
    fallback = OrderServiceClientFallback.class
    // Class that implements this interface and provides fallback responses
    // Used when circuit breaker is open or service is unavailable
)
public interface OrderServiceClient {
    // GET /api/orders/{orderId}
    @GetMapping("/{orderId}")
    OrderDTO getOrderById(@PathVariable('orderId') Long orderId);
    // GET /api/orders?userId=123&status=ACTIVE
    @GetMapping
    List<OrderDTO> getOrdersByUser(
        @RequestParam('userId') Long userId,
        @RequestParam(value = 'status', required = false) String status
    );
    // POST /api/orders with JSON body
    @PostMapping
    OrderDTO createOrder(@RequestBody CreateOrderRequest request);
    // PUT /api/orders/{orderId}
    @PutMapping("/{orderId}")
    OrderDTO updateOrder(
        @PathVariable Long orderId,
        @RequestBody CreateOrderRequest request
    );
    // DELETE /api/orders/{orderId}
    @DeleteMapping("/{orderId}")
    void cancelOrder(@PathVariable Long orderId);
}
```

3.5 Fallback Implementation

```
// ■■ OrderServiceClientFallback.java ■■
// Fallback: returned when order-service is DOWN or circuit is OPEN
@Component
public class OrderServiceClientFallback implements OrderServiceClient {
    private static final Logger log =
        LoggerFactory.getLogger(OrderServiceClientFallback.class);
    @Override
    public OrderDTO getOrderById(Long orderId) {
        log.warn('OrderService unavailable. Returning default for orderId: {}', orderId);
        return new OrderDTO(orderId, 'UNKNOWN', 0.0, 'SERVICE_UNAVAILABLE');
    }
    @Override
    public List<OrderDTO> getOrdersByUser(Long userId, String status) {
        log.warn('OrderService unavailable. Returning empty list for userId: {}', userId);
        return Collections.emptyList();
        // UI shows: 'No orders found' instead of error
    }
    @Override
    public OrderDTO createOrder(CreateOrderRequest request) {
        // Can't create order when service is down – throw business exception
        throw new ServiceUnavailableException(
            'Order service is temporarily unavailable. Please try again in a moment.');
```

3.6 Custom Error Decoder

```
// ■■ CustomFeignErrorDecoder.java ■■
// When target service returns 4xx/5xx, Feign throws FeignException by default.
// Custom ErrorDecoder maps HTTP status codes to meaningful business exceptions.
@Component
public class CustomFeignErrorDecoder implements ErrorDecoder {
    private final ErrorDecoder defaultDecoder = new Default();
    @Override
    public Exception decode(String methodKey, Response response) {
        // methodKey = 'OrderServiceClient#getOrderById(Long)'
        // response.status() = HTTP status code
        // response.body() = raw response body
        switch (response.status()) {
            case 400:
                return new BadRequestException(
```

```

'Invalid request to: ' + methodKey);
case 401:
return new UnauthorizedException(
'Not authorized to call: ' + methodKey);
case 404:
return new ResourceNotFoundException(
'Resource not found when calling: ' + methodKey);
case 503:
// FeignException.retryable() causes Feign to retry
return new RetryableException(
503, 'Service unavailable: ' + methodKey,
Request.HttpMethod.GET, null, response.request());
default:
return defaultDecoder.decode(methodKey, response);
}
}
}

```

3.7 Feign Configuration — Timeout, Logging, Retry

```

// ■■ application.yml – Complete Feign Configuration ■■
feign:
client:
config:
default: # Applies to ALL Feign clients
connect-timeout: 5000 # 5 seconds to open TCP connection
read-timeout: 10000 # 10 seconds to read response
logger-level: BASIC # NONE|BASIC|HEADERS|FULL
error-decoder: com.example.CustomFeignErrorDecoder
retryer: com.example.CustomRetryer # Custom retry logic
order-service: # Only for order-service client (overrides default)
connect-timeout: 2000
read-timeout: 5000
logger-level: FULL # Log full request/response for debugging
payment-service: # Payment might be slow – give more time
connect-timeout: 5000
read-timeout: 30000 # 30 seconds – payment processing takes longer
# Enable circuit breaker integration
circuitbreaker:
enabled: true
# OkHttp is more efficient than default Apache HttpClient
okhttp:
enabled: true
# Feign logging (must also set package log level to DEBUG)
logging:
level:
com.example.clients: DEBUG # Package containing Feign interfaces
# Log Levels explained:
# NONE - No logs (default, use in production for performance)
# BASIC - Method + URL + status code + execution time
# HEADERS - BASIC + all request/response headers

```



```
# FULL - HEADERS + full request/response body (DEV ONLY - security risk in prod)
```

3.8 Request Interceptor — JWT Token Propagation

When a user calls User Service with a JWT token, and User Service calls Order Service via Feign, Order Service also needs the JWT for authorization. The RequestInterceptor automatically propagates it:

```
// ■■ FeignAuthInterceptor.java ■■
@Component
public class FeignAuthInterceptor implements RequestInterceptor {
    @Override
    public void apply(RequestTemplate template) {
        // Get the JWT from the current HTTP request (the one coming INTO User Service)
        ServletRequestAttributes attributes =
            (ServletRequestAttributes) RequestContextHolder.getRequestAttributes();
        if (attributes == null) {
            // No active HTTP request (e.g., scheduled job calling Feign)
            // Handle appropriately – maybe use service account token
            return;
        }
        HttpServletRequest request = attributes.getRequest();
        // Get Authorization header from incoming request
        String authHeader = request.getHeader('Authorization');
        if (authHeader != null && authHeader.startsWith('Bearer ')) {
            // Add it to the outgoing Feign request
            template.header('Authorization', authHeader);
        }
        // Also propagate correlation ID for distributed tracing
        String correlationId = request.getHeader('X-Correlation-Id');
        if (correlationId != null) {
            template.header('X-Correlation-Id', correlationId);
        }
    }
}

// This interceptor runs BEFORE every Feign call automatically.
// No per-call configuration needed.
// Spring auto-detects RequestInterceptor beans and registers them with Feign.
```

★ SENIOR

JWT propagation via RequestInterceptor is ESSENTIAL in secure microservices. Without it, downstream services can't verify who the original caller is. The alternative (micro-service to micro-service trust without JWT) requires network-level security (mTLS via service mesh) — complex to set up without Kubernetes/Istio.

■ PRACTICE QUESTIONS WITH DETAILED ANSWERS

Every question answered in depth — theory, examples, and real-world context.

Q
1

[Conceptual]

What is a declarative REST client? How does Feign differ from RestTemplate?

■ ANSWER:

Declarative means you declare WHAT you want, not HOW to do it. With Feign, you write a Java interface that describes the API contract. Feign's framework generates the implementation.

Comparison:

- **RestTemplate (Imperative):** You write EVERY step — build URL, create HttpEntity, call execute(), check status, extract body, handle exceptions. 15-20 lines per API call.
- **Feign (Declarative):** You write a method signature with annotations. Feign handles URL construction, HTTP call, error handling, deserialization. 1-3 lines per API call.

```
// RestTemplate (imperative - 12 lines):
String url = 'http://order-service/api/orders/' + id;
ResponseEntity<Order> res = restTemplate.getForEntity(url, Order.class);
if(res.getStatusCode() == HttpStatus.OK) return res.getBody();
else throw new RuntimeException();

// Feign (declarative - 1 line interface method):
@GetMapping('/api/orders/{id}')
Order getOrder(@PathVariable Long id); // Feign does everything
```

Other differences:

- **Load Balancing:** RestTemplate needs @LoadBalanced annotation + explicit setup. Feign integrates automatically via Spring Cloud LB.
- **Error Handling:** RestTemplate requires manual status code checks. Feign uses pluggable ErrorDecoder.
- **Maintenance:** Adding a new API? RestTemplate = new boilerplate method. Feign = one new method declaration.

Q
2

[Conceptual]

What does @FeignClient's 'name' attribute specify? What value should it match?

■ ANSWER:

The **name** attribute specifies the service identifier that Feign uses to discover the target service. It must match the spring.application.name of the target service.

```
# order-service application.yml:
spring:
  application:
    name: order-service # ← This is registered in Eureka

# In user-service's Feign client:
@FeignClient(name = 'order-service') # ← Must match above (case-insensitive)
```

What happens when you call a Feign method:

- Feign sees name = 'order-service'
- Passes it to Spring Cloud LoadBalancer: 'Resolve order-service'
- LB queries Eureka: GET /eureka/apps/ORDER-SERVICE → returns [{host1:8082}, {host2:8082}]
- LB picks one instance via Round Robin → returns http://10.0.1.45:8082

- Feign appends the path: `http://10.0.1.45:8082/api/orders/123`
- HTTP call is made

Alternative — URL-based (for external APIs not in Eureka):

```
@FeignClient(name = 'payment-gateway', url = 'https://api.stripe.com')
// Uses URL directly, bypasses Eureka and LoadBalancer
```

Q
3

[Code]

Write a Feign client interface to call a 'product-service' with GET `/products/{id}` and GET `/products?category={cat}`.

■ ANSWER:

Here's the complete implementation:

```
@FeignClient(name = 'product-service', path = '/products')
public interface ProductServiceClient {
    // GET /products/{id}
    @GetMapping("/{id}")
    ProductDTO getProductById(@PathVariable('id') Long id);
    // GET /products?category=Electronics&minPrice=100
    @GetMapping
    List<ProductDTO> getProductsByCategory(
        @RequestParam('category') String category,
        @RequestParam(value = 'minPrice', required = false) Double minPrice,
        @RequestParam(value = 'maxPrice', required = false) Double maxPrice
    );
    // GET /products/in-stock?ids=1,2,3
    @GetMapping('/in-stock')
    List<Long> getInStockProductIds(
        @RequestParam('ids') List<Long> productIds
    );
}
```

Usage in a service:

```
@Service
public class OrderValidationService {
    @Autowired
    private ProductServiceClient productClient;

    public void validateOrder(CreateOrderRequest req) {
        ProductDTO product = productClient.getProductById(req.getProductId());
        if (product == null) throw new ProductNotFoundException();
        if (product.getStock() < req.getQuantity()) throw new InsufficientStockException();
    }
}
```

Q
4

[Conceptual]

What is Feign's default error behavior for 4xx and 5xx responses?

■ ANSWER:

By default, Feign throws a **FeignException** for any non-2xx HTTP response. The exception contains the status code, response body, and the request details.

```
// What Feign throws by default:
```

```
// 404 → FeignException.NotFound (extends FeignClientException)
// 400 → FeignException.BadRequest
// 500 → FeignException.InternalServerError
// 503 → FeignException.ServiceUnavailable

// Catching it:
try {
    order = orderClient.getOrder(123L);
} catch (FeignException.NotFound e) {
    throw new OrderNotFoundException('Order 123 not found');
} catch (FeignException e) {
    log.error('Feign call failed: {} - {}'.format(e.status(), e.getMessage()));
    throw new ServiceException('Order service error');
}
```

Why the default is usually insufficient:

- **FeignException** is not a business exception — it leaks HTTP concerns into business logic
- The caller must know HTTP status codes to handle errors
- No way to return different defaults for different error types

Better approach: Use a custom **ErrorDecoder** to map HTTP errors to business exceptions, and a **fallback** class to return safe defaults when the circuit is open.

Q
5

[Conceptual]

What are the 4 Feign log levels? Which should you use in production?

■ **ANSWER:**

Feign has 4 logging levels, each showing progressively more detail:

- **NONE** — No logging. Zero overhead. Use in production. This is the default.
- **BASIC** — Logs: HTTP method + URL, response status code, execution time. Example:
[OrderServiceClient#getOrder] ---> GET http://order-service/api/orders/123 /
[OrderServiceClient#getOrder] <--- HTTP/1.1 200 in 145ms
- **HEADERS** — Everything in BASIC + all request and response headers. Useful for debugging auth header propagation.
- **FULL** — Everything in HEADERS + request body + response body. Logs the complete JSON. NEVER use in production: it logs sensitive user data (passwords, PII, payment info), creates massive log volume, and degrades performance.

```
# Production:
feign.client.config.default.logger-level: NONE

# Debugging specific service in staging:
feign.client.config.order-service.logger-level: FULL

# Also need package log level:
logging.level.com.example.clients: DEBUG
```

Q
6

[Code]

Write a RequestInterceptor that adds a correlation ID header to every Feign request.

■ **ANSWER:**

Correlation IDs allow you to trace a single user request across multiple microservices in logs:

```
@Component
```

```

public class CorrelationIdInterceptor implements RequestInterceptor {
    // MDC = Mapped Diagnostic Context - thread-local logging context
    private static final String CORRELATION_HEADER = 'X-Correlation-Id';

    @Override
    public void apply(RequestTemplate template) {
        // Try to get from current HTTP request (propagate existing ID)
        String correlationId = getFromCurrentRequest();

        // If no existing ID (e.g., internal background job), generate new one
        if (correlationId == null) {
            correlationId = UUID.randomUUID().toString();
        }

        // Add to outgoing Feign request
        template.header(CORRELATION_HEADER, correlationId);

        // Also set in MDC for this thread's log entries
        MDC.put('correlationId', correlationId);
    }

    private String getFromCurrentRequest() {
        try {
            ServletRequestAttributes attrs = (ServletRequestAttributes)
                RequestContextHolder.getRequestAttributes();
            if (attrs != null) {
                return attrs.getRequest().getHeader(CORRELATION_HEADER);
            }
        } catch (Exception e) {
            // No active request context
        }
        return null;
    }
}

```

With this, every log entry in every service that handles this request will have the same correlationId. Search all logs by correlationId to see the complete journey.

Q
7

[Tricky]

Feign uses name 'order-service'. There are 3 instances in Eureka. What determines which instance Feign calls?

■ **ANSWER:**

The choice of instance is made by **Spring Cloud LoadBalancer**, not Feign itself. Feign's role is only to know the service name; it delegates instance selection to the load balancer.

The selection process:

- **Step 1:** Feign gets service name 'order-service'
- **Step 2:** Feign calls `BlockingLoadBalancerClient.choose('order-service')`
- **Step 3:** Spring Cloud LB has cached list: [Instance1: 10.0.1.5:8082, Instance2: 10.0.1.6:8082, Instance3: 10.0.1.7:8082]
- **Step 4:** Default strategy (Round Robin) picks the next in rotation
- **Step 5:** Returns selected ServiceInstance to Feign
- **Step 6:** Feign builds URL: `http://10.0.1.5:8082/api/orders/123`

The rotation is maintained per service. Request 1 → Instance 1, Request 2 → Instance 2, Request 3 → Instance 3, Request 4 → Instance 1...

Important: The cache of instances is refreshed every 30 seconds from Eureka. There's a brief window where a new instance isn't visible, or a dead instance is still listed.

Q
8

[Scenario]

You're calling Payment Service from Order Service using Feign. Payment Service takes 30 seconds to respond. How do you handle this?

■ **ANSWER:**

This is a classic distributed systems problem. A 30-second response time will exhaust Order Service's thread pool if not handled.

Complete solution:

Step 1 — Set appropriate timeout:

```
feign:
  client:
  config:
    payment-service:
      connect-timeout: 5000
      read-timeout: 35000 # Give 35 seconds (slightly more than max expected)
```

Step 2 — Add Circuit Breaker:

```
@FeignClient(name = 'payment-service', fallback = PaymentFallback.class)
public interface PaymentClient {
    @PostMapping('/api/payments')
    PaymentResult processPayment(@RequestBody PaymentRequest req);
}
```

Step 3 — Configure circuit breaker:

```
resilience4j:
  circuitbreaker:
    instances:
      payment-service:
        slidingWindowSize: 10
        failureRateThreshold: 50 # Open after 50% failure rate
        waitDurationInOpenState: 60s
        permittedNumberOfCallsInHalfOpenState: 3
```

Step 4 — Consider async processing:

If 30 seconds is acceptable (e.g., complex payment processing), consider making it async: Order Service publishes PaymentRequested event → Payment Service processes it → publishes PaymentCompleted event → Order Service updates status. This frees the thread immediately.

Q
9

[Tricky]

Can you use @FeignClient with a URL instead of a service name? When would you do this?

■ **ANSWER:**

Yes. When you specify url in @FeignClient, Feign bypasses Eureka and Spring Cloud LoadBalancer entirely and calls the URL directly.

```
// External third-party API (not in Eureka):
```

```
@FeignClient(name = 'stripe-api', url = 'https://api.stripe.com/v1')
public interface StripeClient {
    @PostMapping('/charges')
    ChargeResponse createCharge(@RequestBody ChargeRequest request);
}
```

When to use URL-based Feign clients:

- **External APIs:** Calling Stripe, Razorpay, Twilio, SendGrid — they're not in your Eureka registry
- **Legacy services:** Old services with fixed IP/hostname that don't use Eureka
- **Testing/mocking:** WireMock server URL during integration tests
- **Direct calls:** Bypassing load balancing when you need a specific instance

Note: Even with url specified, you still need a name (required field). But the name is not used for service discovery — it's just for identification in logs and configuration.

For environment-specific URLs: externalize it to config:

```
@FeignClient(name = 'payment', url = '${payment.gateway.url}')
// payment.gateway.url=https://api.stripe.com in prod
// payment.gateway.url=http://localhost:9999 in dev (WireMock)
```

API Gateway — Spring Cloud Gateway

The single intelligent front door to your microservices ecosystem.

4.1 Why API Gateway? The Problems Without One

Without an API Gateway, here's what your architecture looks like:

// ■■ Without API Gateway – The Chaos ■■

Client (mobile app) needs to show a product page:

→ Call User Service: GET http://user-service:8081/profile

→ Call Product Service: GET http://product-service:8083/products/456

→ Call Review Service: GET http://review-service:8085/reviews?productId=456

→ Call Recommendation: GET http://recommendation-service:8087/similar/456

→ Call Inventory Service: GET http://inventory-service:8086/stock/456

Problems:

1. Client knows 5 different URLs - tight coupling
2. Client handles auth for all 5 services separately
3. 5 network round trips on a slow mobile network
4. CORS configuration duplicated in all 5 services
5. Rate limiting logic duplicated everywhere
6. SSL termination needed at each service
7. Change a service URL? Update ALL clients.

With an API Gateway:

// ■■ With API Gateway – The Clean Architecture ■■

Client calls ONE URL: https://api.myapp.com/

→ GET /api/users/profile → routes to User Service

→ GET /api/products/456 → routes to Product Service

→ GET /api/reviews?pid=456 → routes to Review Service

Benefits:

1. Client knows ONE URL - loose coupling
2. Auth handled ONCE at the gateway for all services
3. Response aggregation can reduce round trips
4. CORS handled once at gateway
5. Rate limiting in one place
6. SSL termination at gateway - services use HTTP internally
7. Change internal service routing? Update ONLY the gateway.

4.2 Spring Cloud Gateway Architecture

Spring Cloud Gateway (SCG) is built on **Spring WebFlux** (Project Reactor), making it fully **non-blocking and reactive**. This means it can handle thousands of concurrent connections

without proportionally increasing thread count.

Compare: Servlet-based (traditional) vs Reactive:

Aspect	Servlet (Spring MVC)	Reactive (Spring WebFlux)
Thread Model	1 thread per request — blocks until response	Small thread pool — threads never block, handle events
Concurrency	~200-500 concurrent requests per node	Thousands of concurrent connections per node
Memory	Each blocked thread = ~256KB stack memory	No blocking = much lower memory per connection
When to use	Standard business logic services	Gateway, API aggregation, streaming, proxies

4.3 Core Building Blocks

4.3.1 Route

A Route is the basic unit of configuration in Spring Cloud Gateway. Each route has:

- **ID**: Unique identifier for this route (e.g., 'order-service-route')
- **URI**: Destination — where to forward the request (e.g., lb://order-service or https://external-api.com)
- **Predicates**: Conditions — when to apply this route
- **Filters**: Transformations — what to do to the request/response

4.3.2 Predicates

Predicates are conditions evaluated against the incoming HTTP request. If ALL predicates of a route match, that route is applied.

// ■■ Predicate Examples ■■

Common Predicates:

Path-based routing (most common):

- Path=/api/orders/** # Matches any path starting with /api/orders/

- Path=/api/users/{id} # Path with template variable

Method-based:

- Method=GET,POST # Only GET and POST requests

Header-based:

- Header=X-Request-Version, v2 # Header exists and equals 'v2'

- Header=Authorization # Header just needs to exist

Time-based:

- Before=2025-12-31T23:59:59Z # Only before this date

- After=2024-01-01T00:00:00Z # Only after this date

- Between=2024-01-01T00:00:00Z, 2025-01-01T00:00:00Z

Query parameter:

- Query=debug # Request has ?debug parameter

- Query=userId, \d+ # ?userId=123 (regex match)

Client IP:

```
- RemoteAddr=192.168.1.0/24 # Only from this IP range
# Multiple predicates = AND logic:
predicates:
- Path=/api/payments/**
- Method=POST # Only POST to /api/payments/**
# Both conditions must be true
```

4.3.3 Filters

Filters transform requests (before forwarding) or responses (after receiving). They run in a chain — each filter can modify and pass to the next.

```
// ■■ Filter Examples ■■
# Pre-filters (run before forwarding to downstream service):
- AddRequestHeader=X-Source, gateway # Add header to request
- RemoveRequestHeader=Authorization # Remove header (e.g., before logging)
- AddRequestParameter=format, json # Add query param
- SetPath=/api/{segment} # Change the path
- StripPrefix=1 # Remove first path segment
# /api/orders/123 → /orders/123
- RewritePath=/old-path/(?<seg>.*), /new/${seg} # Regex rewrite
- RequestRateLimiter # Rate limit
- CircuitBreaker # Circuit breaker
# Post-filters (run after receiving response from downstream):
- AddResponseHeader=X-Response-Time, 200ms # Add to response
- RemoveResponseHeader=X-Internal-Token # Remove sensitive internal headers
- SetStatus=200 # Override status code
# Special:
- Retry=3 # Retry up to 3 times on 5xx
- SetRequestHost=internal.service.com # Change Host header
```

4.4 Complete Gateway Configuration

```
// application.yml - Complete Gateway Configuration
server:
port: 8080
spring:
application:
name: api-gateway
cloud:
gateway:
# Global CORS configuration
globalcors:
cors-configurations:
['/**']:
allowedOrigins: '*'
allowedMethods: 'GET,POST,PUT,DELETE,OPTIONS'
allowedHeaders: '*'
maxAge: 3600
routes:
# USER SERVICE
- id: user-service-route
```

[illegible]

```
defaultZone: http://localhost:8761/eureka/
```

4.5 Custom Global Filter

```
// ■■ RequestLoggingFilter.java – Global Filter ■■
// Global Filter: runs for EVERY request regardless of route
@Component
@Order(-1) // -1 = highest priority, runs first in pre, last in post
public class RequestLoggingFilter implements GlobalFilter {
    private static final Logger log =
        LoggerFactory.getLogger(RequestLoggingFilter.class);

    @Override
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {
        ServerHttpRequest request = exchange.getRequest();
        // Generate unique ID for this request
        String requestId = UUID.randomUUID().toString().substring(0, 8);
        long startTime = System.currentTimeMillis();
        // PRE-FILTER: runs before forwarding
        log.info('[{}] --> {} {} | IP: {}',
            requestId,
            request.getMethod(),
            request.getURI().getPath(),
            request.getRemoteAddress()
        );
        // Mutate exchange to add request ID header (passed to downstream)
        ServerWebExchange mutated = exchange.mutate()
            .request(req -> req.header('X-Request-Id', requestId))
            .build();
        // chain.filter continues to next filter, then to downstream service
        return chain.filter(mutated)
            .then(Mono.fromRunnable(() -> {
                // POST-FILTER: runs AFTER response received
                long duration = System.currentTimeMillis() - startTime;
                log.info('[{}] <-- {} | {}ms',
                    requestId,
                    exchange.getResponse().getStatusCode(),
                    duration
                );
            }));
    }
}
```

4.6 Authentication Filter

```
// ■■ JwtAuthFilter.java – Complete JWT Auth at Gateway ■■
@Component
public class JwtAuthFilter implements GlobalFilter, Ordered {
    @Autowired private JwtUtil jwtUtil;
    private static final List<String> PUBLIC_PATHS = List.of(
        '/api/auth/login', '/api/auth/register',

```

```

'/api/auth/refresh', '/actuator/health'
);
@Override
public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {
    String path = exchange.getRequest().getURI().getPath();
    // Skip auth for public endpoints
    boolean isPublic = PUBLIC_PATHS.stream().anyMatch(path::startsWith);
    if (isPublic) return chain.filter(exchange);
    // Extract JWT from Authorization header
    List<String> authHeaders = exchange.getRequest()
        .getHeaders().get(HttpHeaders.AUTHORIZATION);
    if (authHeaders == null || authHeaders.isEmpty()) {
        return unauthorized(exchange, 'Missing Authorization header');
    }
    String token = authHeaders.get(0);
    if (!token.startsWith('Bearer ')) {
        return unauthorized(exchange, 'Invalid token format');
    }
    String jwt = token.substring(7);
    if (!jwtUtil.isValid(jwt)) {
        return unauthorized(exchange, 'Invalid or expired token');
    }
    // Extract user info and add to headers for downstream services
    String userId = jwtUtil.extractUserId(jwt);
    String role = jwtUtil.extractRole(jwt);
    ServerWebExchange mutated = exchange.mutate()
        .request(req -> req
            .header('X-User-Id', userId)
            .header('X-User-Role', role)
            .header('X-Authenticated', 'true')
        )
        .build();
    return chain.filter(mutated);
}

private Mono<Void> unauthorized(ServerWebExchange exchange, String message) {
    exchange.getResponse().setStatusCode(HttpStatus.UNAUTHORIZED);
    exchange.getResponse().getHeaders()
        .setContentType(MediaType.APPLICATION_JSON);
    String body = '{"error":"' + message + '"}';
    DataBuffer buffer = exchange.getResponse().bufferFactory()
        .wrap(body.getBytes());
    return exchange.getResponse().writeWith(Mono.just(buffer));
}
@Override
public int getOrder() { return -2; } // Before logging filter
}

```

★ SENIOR

By extracting `userId` and `role` from JWT at the Gateway and adding them as headers, downstream services DON'T need JWT libraries. They just read `X-User-Id` header. This simplifies services. But: services must trust these headers. They're behind the gateway — direct external access should be blocked by firewall rules.

4.7 Rate Limiting

```
// ■■ Rate Limiting with Redis Token Bucket ■■
# Rate limiting requires Redis for distributed state (shared across gateway
instances)
spring:
data:
redis:
host: localhost
port: 6379
cloud:
gateway:
routes:
- id: order-service-rate-limited
uri: lb://order-service
predicates:
- Path=/api/orders/**
filters:
- name: RequestRateLimiter
args:
# Token Bucket algorithm:
redis-rate-limiter.replenishRate: 10
# 10 tokens added per second to the bucket
redis-rate-limiter.burstCapacity: 20
# Bucket can hold max 20 tokens
# Allows bursts of up to 20 requests
redis-rate-limiter.requestedTokens: 1
# Each request costs 1 token
key-resolver: '#{@userKeyResolver}'
# Rate limit PER USER (not per IP)
# Different users get separate buckets
# Define the key resolver bean:
# In your @Configuration class:
# @Bean
# KeyResolver userKeyResolver() {
# return exchange -> {
# // Rate limit per user ID (extracted from JWT by auth filter)
# String userId = exchange.getRequest().getHeaders().getFirst('X-User-Id');
# return Mono.just(userId != null ? userId : 'anonymous');
# };
# }
# When rate limit exceeded: 429 Too Many Requests
# Response headers:
# X-RateLimit-Remaining: 0
```

```
# X-RateLimit-Replenish-Rate: 10
```

```
# X-RateLimit-Burst-Capacity: 20
```

■ PRACTICE QUESTIONS WITH DETAILED ANSWERS

Every question answered in depth — theory, examples, and real-world context.

Q
1

[Conceptual]

What is an API Gateway? What problems does it solve in a microservices architecture?

■ ANSWER:

An **API Gateway** is a server that acts as the single entry point for all client requests in a microservices architecture. Instead of clients calling individual services directly, they call the gateway, which routes requests to appropriate services.

Problems it solves:

- **Client Coupling:** Without a gateway, clients must know the URL of every microservice. Add a new service? All clients need updating. With a gateway, clients know ONE URL.
- **Cross-Cutting Concerns Duplication:** Authentication, logging, rate limiting, CORS — without a gateway, each of 50 services implements these independently. Inconsistent and hard to maintain. Gateway centralizes them.
- **Network Ch chattiness:** A mobile app loading one screen might need to call 5 services — 5 network round trips on slow mobile network. A gateway can aggregate responses (Backend for Frontend pattern).
- **Protocol Translation:** External clients use HTTPS/REST. Internal services might use gRPC, WebSocket, or HTTP. Gateway translates protocols.
- **Security Boundary:** Internal services don't need to be exposed to the internet. Only the gateway has a public IP. All other services are on a private network. Security surface area reduced dramatically.

What Spring Cloud Gateway provides specifically: reactive routing, route predicates, filter chain, circuit breakers, rate limiting (Redis), load balancing integration, WebSocket support, and Actuator monitoring.

Q
2

[Conceptual]

What are the three core components of Spring Cloud Gateway? Explain each.

■ ANSWER:

Spring Cloud Gateway is built on three core abstractions:

1. Route:

The fundamental building block. A route is a mapping rule: when THIS condition is true, send the request HERE after doing THESE things. Each route has: an ID (for logging/metrics), a destination URI (lb://service-name or https://external.com), predicates (matching conditions), and filters (transformations).

```
- id: my-route
uri: lb://my-service
predicates:
```

```
- Path=/api/**  
filters:  
- AddRequestHeader=X-Source, gateway
```

2. Predicate:

A boolean function evaluated against the incoming `ServerWebExchange` (the request + response wrapper in reactive terms). If the predicate returns true, this route is selected. Multiple predicates on one route are ANDed together. Spring Cloud Gateway has 10+ built-in predicates: Path, Method, Header, Query, Cookie, Before, After, Between, Host, RemoteAddr.

3. Filter:

A function that modifies the request (pre-filter) or response (post-filter). Filters form a chain — each filter processes the exchange and passes it to the next. Global filters run for every route. Route filters run only for matching routes. 30+ built-in filters available. Custom filters implement `GlobalFilter` or `GatewayFilter` interfaces.

Q
3

[Conceptual]

Why does Spring Cloud Gateway use WebFlux instead of Spring MVC? What does non-blocking mean?

■ ANSWER:

Spring Cloud Gateway uses WebFlux because a gateway's primary job is **I/O proxying** — receiving requests, forwarding them, and returning responses. This is I/O-intensive, not CPU-intensive.

Traditional (blocking) I/O model:

- Thread 1 receives request → forwards to service → BLOCKS, waiting for response → returns response
- While Thread 1 is blocked, it can't do other work
- For 1,000 concurrent requests: need 1,000 threads
- Each thread uses 256KB-1MB of stack memory → 1GB+ memory for 1,000 threads

Non-blocking (reactive) I/O model:

- Thread 1 receives request → starts forwarding → registers a callback → Thread 1 is FREE to handle other requests
- When response arrives → callback is triggered → response returned to client
- 4-8 threads can handle thousands of concurrent connections
- Memory usage: dramatically lower

For a gateway handling 10,000 concurrent requests: blocking model needs 10,000 threads (~10GB RAM). Non-blocking model needs ~8 threads (~8MB). This is why gateways, proxies, and streaming services use reactive.

Critical warning: Never add `spring-boot-starter-web` to a Spring Cloud Gateway project. They use different HTTP stacks (Tomcat Servlet vs Netty Reactive) that conflict. You'll get a confusing startup error.

Q
4

[Code]

Write a YAML configuration for a Gateway route that routes `/api/products/**` to `product-service`, adds a request header, and retries up to 3 times on 503.

■ ANSWER:

Complete YAML configuration:

```
spring:
cloud:
gateway:
routes:
- id: product-service-route
uri: lb://product-service
# lb:// = load balanced via Spring Cloud LB + Eureka
predicates:
- Path=/api/products/**
# Matches: /api/products, /api/products/123, /api/products/123/reviews
filters:
- AddRequestHeader=X-Source, api-gateway
# Adds header to every forwarded request
# Downstream service sees: X-Source: api-gateway
- name: Retry
args:
retries: 3
# Retry up to 3 times (total 4 attempts)
statuses: SERVICE_UNAVAILABLE
# Only retry on 503 SERVICE_UNAVAILABLE
# NOT on 404 (resource not found - no point retrying)
# NOT on 400 (bad request - no point retrying)
methods: GET,HEAD
# ONLY retry GET and HEAD (safe, idempotent methods)
# NEVER retry POST - could create duplicate orders!
backoff:
firstBackoff: 100ms
maxBackoff: 500ms
factor: 2
# Exponential backoff: 100ms, 200ms, 400ms
```

Q
5

[Code]

Write a `GlobalFilter` that logs every incoming request's method, path, and response time.

■ ANSWER:

Here's a complete, production-grade logging filter:

```
@Component
@Order(-1)
public class RequestLoggingFilter implements GlobalFilter {
    private final Logger log = LoggerFactory.getLogger(this.getClass());

    @Override
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {
        ServerHttpRequest req = exchange.getRequest();
        String requestId = UUID.randomUUID().toString().substring(0, 8);
        long start = System.currentTimeMillis();

        // PRE: log incoming request
        log.info('[{}] --> {} {} from {}',
            requestId, req.getMethod(), req.getPath(),
```

```

req.getRemoteAddress());

// Mutate: add requestId header for downstream tracing
ServerWebExchange mutated = exchange.mutate()
    .request(r -> r.header('X-Request-Id', requestId))
    .build();

return chain.filter(mutated).then(Mono.fromRunnable(() -> {
    // POST: log response after completion
    log.info('[{}] <-- {} in {}ms',
        requestId,
        exchange.getResponse().getStatusCode(),
        System.currentTimeMillis() - start);
})));
}
}

```

Q
6

[Conceptual]

What is the Discovery Locator in Spring Cloud Gateway? What routes does it auto-create?

■ **ANSWER:**

Discovery Locator is a feature of Spring Cloud Gateway that **automatically creates routes for every service registered in Eureka**, without any manual route configuration.

```

spring:
  cloud:
    gateway:
      discovery:
        locator:
          enabled: true
          lower-case-service-id: true # Convert ORDER-SERVICE → order-service

```

With this enabled, if Eureka has these services: ORDER-SERVICE, USER-SERVICE, PAYMENT-SERVICE, it automatically creates:

- GET /order-service/** → forwards to ORDER-SERVICE
- GET /user-service/** → forwards to USER-SERVICE
- GET /payment-service/** → forwards to PAYMENT-SERVICE

The route is: Path=/[service-name]/** → lb://[service-name]

When to use Discovery Locator vs Manual Routes:

- **Use Discovery Locator:** Development, internal tools, admin dashboards — quick setup, zero config
- **Use Manual Routes:** Production APIs — you need path rewriting (hide internal service names), per-route filters (rate limiting, auth), and the URL structure you expose externally differs from internal service names

Example: You want /api/orders/** externally but internally it's http://order-service/api/orders/**. Discovery Locator would expose it as /order-service/**. Manual route gives you control.

Q
7

[Tricky]

A request passes through 3 global filters and 2 route filters. What is the order of execution for pre-filters and post-filters?

■ **ANSWER:**

This is a critical concept — the filter execution order in Spring Cloud Gateway follows a specific pattern.

Assume: GlobalFilter1 (order=-2), GlobalFilter2 (order=-1), GlobalFilter3 (order=0), RouteFilter1 (first in list), RouteFilter2 (second in list)

Pre-filter order (highest priority first):

- 1. GlobalFilter1 pre (order=-2, highest priority)
- 2. GlobalFilter2 pre (order=-1)
- 3. GlobalFilter3 pre (order=0)
- 4. RouteFilter1 pre
- 5. RouteFilter2 pre
- 6. → REQUEST FORWARDED TO DOWNSTREAM SERVICE →

Post-filter order (REVERSE):

- 7. RouteFilter2 post (innermost first)
- 8. RouteFilter1 post
- 9. GlobalFilter3 post
- 10. GlobalFilter2 post
- 11. GlobalFilter1 post (outermost last)

This follows the same pattern as Java Servlet filters — it's a stack. The first filter to run in pre-processing is the last to run in post-processing. This is important: your auth filter (order=-2) should run pre, but its post cleanup runs LAST after the response.

Q
8

[Design]

Design an API Gateway configuration for an app with User Service, Order Service, Product Service, and Payment Service — including authentication, logging, and rate limiting.

■ **ANSWER:**

Here's a complete production-grade gateway design:

Architecture decision: Global filters for cross-cutting concerns. Route-specific filters for per-service policies.

Global Filters (apply to all routes):

- Order -3: RequestLoggingFilter — logs request/response with timing
- Order -2: JwtAuthFilter — validates JWT, adds X-User-Id header
- Order -1: CorrelationIdFilter — ensures X-Correlation-Id propagated

Route configurations:

```
routes:
- id: user-service
uri: lb://user-service
predicates: [Path=/api/users/**]
# Auth already done globally. User-specific: no extra rate limit
- id: order-service
uri: lb://order-service
predicates: [Path=/api/orders/**]
filters:
- name: CircuitBreaker
args: {name: orderCB, fallbackUri: 'forward:/fallback'}
```

```
- name: Retry
args: {retries: 2, statuses: SERVICE_UNAVAILABLE, methods: GET}

- id: product-service
uri: lb://product-service
predicates: [Path=/api/products/**]
filters:
- name: RequestRateLimiter
args:
redis-rate-limiter.replenishRate: 50 # Products are read-heavy
redis-rate-limiter.burstCapacity: 100
key-resolver: '#{@ipKeyResolver}'

- id: payment-service
uri: lb://payment-service
predicates:
- Path=/api/payments/**
- Method=POST,PUT # Only write operations through gateway
# No retry on payments! Double charge risk
```

Spring Cloud Load Balancer

Distributing traffic intelligently across service instances for resilience and performance.

5.1 What Is Load Balancing?

Load balancing is the practice of distributing incoming requests across multiple instances of a service to ensure no single instance is overwhelmed, and to maximize availability when instances fail.

Without load balancing with multiple instances:

```
// ■■ Without vs With Load Balancing ■■
# 3 instances of Order Service running:
# order-service-1: 10.0.1.5:8082 (handling 500 req/sec - OVERLOADED)
# order-service-2: 10.0.1.6:8082 (idle - 0 req/sec)
# order-service-3: 10.0.1.7:8082 (idle - 0 req/sec)
# Without LB: all requests hardcoded to instance-1
# Instance-1 crashes → ALL requests fail
# With Load Balancer:
# order-service-1: 167 req/sec
# order-service-2: 167 req/sec
# order-service-3: 167 req/sec
# Instance-1 crashes → LB detects, routes to 2 & 3
# Users are unaffected (except brief retry if using stale route)
```

5.2 Client-Side vs Server-Side Load Balancing

5.2.1 Server-Side Load Balancing (Traditional)

In server-side LB, a dedicated infrastructure component (hardware or software) sits between clients and servers. It receives all traffic and distributes it.

```
// ■■ Server-Side Load Balancing ■■
Client → [AWS ALB / Nginx / HAProxy] → [Instance1]
→ [Instance2]
→ [Instance3]
# Client makes ONE call to the LB address.
# LB decides which instance to use.
# Client doesn't know about individual instances.
```

- **Pros:** Centralized, client-agnostic, works for any client type (browser, mobile, microservice)
- **Cons:** Single point of failure (must be HA), additional network hop, additional infrastructure cost
- **Examples:** AWS ALB, Google Cloud LB, Nginx, HAProxy, F5

5.2.2 Client-Side Load Balancing (Spring Cloud)

In client-side LB, the load balancing logic runs INSIDE the calling service (the client). The client fetches all available instances from Eureka, caches them, and chooses one for each request.

```
// ■■ Client-Side Load Balancing ■■
```

```
Service A (client):
```

1. Fetches instance list from Eureka: [Instance1, Instance2, Instance3]
2. Caches locally for 30 seconds
3. For each request, applies Round Robin to pick an instance
4. Calls chosen instance directly

```
Service A → [Instance1] (request 1)
```

```
Service A → [Instance2] (request 2)
```

```
Service A → [Instance3] (request 3)
```

```
Service A → [Instance1] (request 4)
```

```
# No separate LB infrastructure needed!
```

```
# Spring Cloud LoadBalancer provides this logic.
```

- **Pros:** No additional infrastructure, lower latency (no extra hop), each client can apply different strategy
- **Cons:** LB logic distributed in each service, stale cache issue (up to 30s delay in discovering new/removed instances)

5.3 Load Balancing Strategies

■ Round Robin (Default)

Requests are distributed in a fixed rotation: R1→I1, R2→I2, R3→I3, R4→I1, R5→I2... Simple and effective when all instances are equally powerful. PROBLEM: If one instance is slower (old hardware), it still gets the same share of traffic. Use case: Homogeneous infrastructure (all instances same spec).

■ Random

Randomly selects an instance for each request. At high request volumes, distribution becomes roughly equal by the law of large numbers. Simpler to implement than Round Robin. Use case: When randomness is preferred over strict rotation (avoids synchronized behavior).

■ Weighted Response Time

Tracks response time for each instance. Faster instances get proportionally more traffic. If Instance 1 responds in 50ms and Instance 2 in 200ms, Instance 1 gets 4x more traffic. Must be implemented as a custom LoadBalancer in Spring Cloud. Use case: Mixed hardware environments.

■ Zone-Aware Round Robin

Prefers instances in the same availability zone (AZ) as the caller. Reduces cross-AZ network latency (can be 10-50ms) and cross-AZ data transfer costs. Only falls back to other AZs when local AZ has no healthy instances. Use case: Multi-region/multi-AZ production

deployments.

■ Least Connections

Routes to the instance with the fewest active connections. Optimal when requests have highly variable processing times. Not built-in to Spring Cloud LB — requires custom implementation. Use case: Services with long-running operations (file processing, video transcoding).

5.4 Setting Up Spring Cloud LoadBalancer

5.4.1 Dependencies

```
// ■■ pom.xml ■■
<!-- Spring Cloud LoadBalancer is included in: -->
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-loadbalancer</artifactId>
</dependency>
<!-- Also included TRANSITIVELY when you add: -->
<!-- spring-cloud-starter-openfeign (Feign auto-includes LB) -->
<!-- spring-cloud-starter-gateway (Gateway auto-includes LB) -->
<!-- So for Feign or Gateway projects, you usually don't add it explicitly -->
<!-- IMPORTANT: Remove spring-cloud-starter-netflix-ribbon if present! -->
<!-- Ribbon is deprecated and conflicts with Spring Cloud LoadBalancer -->
```

5.4.2 Load Balanced RestTemplate

```
// ■■ Load Balanced RestTemplate Setup ■■
@Configuration
public class AppConfig {
    @Bean
    @LoadBalanced
    // @LoadBalanced adds a special interceptor to RestTemplate.
    // This interceptor intercepts calls to http://service-name/... URLs,
    // resolves 'service-name' via Spring Cloud LB + Eureka,
    // and replaces it with the actual IP:port.
    public RestTemplate loadBalancedRestTemplate() {
        return new RestTemplate();
    }
    // WARNING: If you also need a non-load-balanced RestTemplate
    // (e.g., for calling external APIs by IP), create a SEPARATE bean:
    @Bean
    // @Primary ← Don't make this primary if LB is primary
    public RestTemplate plainRestTemplate() {
        return new RestTemplate();
    }
}
@Service
public class OrderService {
    @Autowired
```

```

@Qualifier('loadBalancedRestTemplate')
private RestTemplate restTemplate;

public List<Order> getUserOrders(Long userId) {
    // 'order-service' resolved via Eureka by @LoadBalanced interceptor
    // Automatically uses Round Robin across all instances
    String url = 'http://order-service/api/orders?userId=' + userId;
    return restTemplate.getForObject(url, List.class);
}
}

```

5.4.3 Load Balanced WebClient (Reactive)

```

// ■■ Load Balanced WebClient Setup ■■
@Configuration
public class WebClientConfig {
    @Bean
    @LoadBalanced
    public WebClient.Builder webClientBuilder() {
        return WebClient.builder()
            .defaultHeader(HttpHeaders.CONTENT_TYPE, MediaType.APPLICATION_JSON_VALUE)
            .filter(ExchangeFilterFunction.ofRequestProcessor(req -> {
                // Add correlation ID to all outgoing calls
                return Mono.just(ClientRequest.from(req)
                    .header('X-Correlation-Id', UUID.randomUUID().toString()))
                    .build());
            }));
    }
}

@Service
public class ReactiveInventoryService {
    @Autowired
    private WebClient.Builder webClientBuilder;

    public Mono<Integer> getStockCount(Long productId) {
        return webClientBuilder.build()
            .get()
            .uri('http://inventory-service/api/inventory/' + productId)
            // 'inventory-service' resolved via Eureka + LB
            .retrieve()
            .onStatus(HttpStatus::is4xxClientError,
                response -> Mono.error(new ProductNotFoundException()))
            .onStatus(HttpStatus::is5xxServerError,
                response -> Mono.error(new ServiceException()))
            .bodyToMono(Integer.class)
            .timeout(Duration.ofSeconds(5))
            .onErrorReturn(0); // Return 0 stock on timeout/error
    }
}

```

5.5 Custom Load Balancing Strategy

```

// ■■ Custom Load Balancer Implementation ■■

```



```

// Custom strategy: prefer instances tagged with metadata 'zone=primary'
public class PrimaryZoneLoadBalancer implements ReactorServiceInstanceLoadBalancer
{
    private final String serviceId;
    private final ObjectProvider<ServiceInstanceListSupplier> supplierProvider;
    public PrimaryZoneLoadBalancer(
        ObjectProvider<ServiceInstanceListSupplier> supplierProvider,
        String serviceId) {
        this.serviceId = serviceId;
        this.supplierProvider = supplierProvider;
    }
    @Override
    public Mono<Response<ServiceInstance>> choose(Request request) {
        return supplierProvider.getIfAvailable(NoopServiceInstanceListSupplier::new)
            .get(request)
            .next()
            .map(instances -> selectInstance(instances));
    }
    private Response<ServiceInstance> selectInstance(List<ServiceInstance> instances)
    {
        if (instances.isEmpty()) {
            return new EmptyResponse();
        }
        // Prefer instances with metadata 'zone=primary'
        List<ServiceInstance> primaryInstances = instances.stream()
            .filter(i -> 'primary'.equals(i.getMetadata().get('zone')))
            .collect(Collectors.toList());
        // Fall back to any instance if no primary available
        List<ServiceInstance> candidates = primaryInstances.isEmpty() ?
            instances : primaryInstances;
        // Round robin within candidates
        int index = ThreadLocalRandom.current().nextInt(candidates.size());
        return new DefaultResponse(candidates.get(index));
    }
}

// Register for specific service:
@Configuration
@LoadBalancerClient(name = 'order-service',
    configuration = OrderServiceLBConfig.class)
public class OrderServiceLBConfig {
    @Bean
    public ReactorServiceInstanceLoadBalancer orderServiceLoadBalancer(
        ObjectProvider<ServiceInstanceListSupplier> supplier,
        Environment env) {
        String name = env.getProperty(LoadBalancerClientFactory.PROPERTY_NAME);
        return new PrimaryZoneLoadBalancer(supplier, name);
    }
}

```

5.6 Health Check Integration

```
// ■■ Health Check Based Load Balancing ■■
# application.yml – Health check based instance filtering
spring:
cloud:
loadbalancer:
health-check:
enabled: true
initial-delay: 3s
# Wait 3 seconds before first health check
interval: 25s
# Check every 25 seconds
path:
default: /actuator/health
# Health check endpoint on each instance
configurations: health-check
# Use health-check filtering (removes UNHEALTHY instances from LB pool)
management:
endpoints:
web:
exposure:
include: health,info
endpoint:
health:
show-details: always
# What this does:
# Spring Cloud LB periodically calls /actuator/health on each instance
# If status is DOWN or connection refused → removes from load balancing pool
# No more routing to unhealthy instances
# Instance recovers and health returns UP → automatically added back
```

★ SENIOR

In production, layer multiple resilience mechanisms: (1) Eureka heartbeat removes dead instances from registry. (2) Spring Cloud LB health checks remove unhealthy instances from LB pool. (3) Resilience4j circuit breaker stops calling services with high failure rate. (4) Retry with exponential backoff on transient failures. These form a defense-in-depth strategy for resilient microservices.

■ PRACTICE QUESTIONS WITH DETAILED ANSWERS

Every question answered in depth — theory, examples, and real-world context.

Q
1

[Conceptual]

What is load balancing? Why is it necessary in microservices?

■ ANSWER:

Load balancing is the distribution of incoming requests across multiple instances of a service to maximize throughput, minimize response time, and ensure high availability.

Why necessary in microservices:

- **Multiple Instances:** For availability and performance, you run multiple instances of each service. Without LB, all traffic goes to one instance while others are idle.
- **Instance Failures:** In cloud environments, instances crash, get replaced, or are terminated. Without LB detecting this, requests keep going to dead instances.
- **Auto-scaling:** When load spikes, new instances are added. Without LB, these new instances receive zero traffic.

Concrete example: Order Service runs 3 instances during normal hours. During a sale, it scales to 10 instances. Without LB, only the original 3 instances are ever called. The extra 7 instances are completely unused. With LB, all 10 share the load equally.

Q
2

[Conceptual]

What is the difference between client-side and server-side load balancing?

■ **ANSWER:**

The fundamental difference is **where the load balancing decision is made:**

Server-Side LB:

- A dedicated server (AWS ALB, Nginx, HAProxy) receives all traffic
- Client sends request to LB's address — doesn't know about individual instances
- LB queries a service registry or uses static config to pick an instance
- LB forwards the request
- Additional network hop but completely transparent to clients

Client-Side LB (Spring Cloud):

- Calling service (client) fetches instance list from Eureka
- Client has Spring Cloud LB library that picks an instance
- Client calls the chosen instance directly
- No extra hop, but LB logic distributed across all services

Spring Cloud uses client-side LB because it integrates naturally with Eureka. When using Kubernetes, you often use server-side LB (K8s Service) instead.

Q
3

[Conceptual]

What load balancing strategy does Spring Cloud LoadBalancer use by default?

■ **ANSWER:**

Round Robin is the default strategy in Spring Cloud LoadBalancer (replacing Ribbon's random-based default).

Round Robin operation with 3 instances:

Instances: [A: 10.0.1.5:8082, B: 10.0.1.6:8082, C: 10.0.1.7:8082]

Request 1 → Instance A

Request 2 → Instance B

Request 3 → Instance C

Request 4 → Instance A (cycle repeats)

Request 5 → Instance B

Request 6 → Instance C

...

The counter is maintained per service in an AtomicInteger. Thread-safe via atomic increment + modulo operation:

```
int pos = this.position.incrementAndGet() & Integer.MAX_VALUE;
ServiceInstance instance = instances.get(pos % instances.size());
```

When to change from Round Robin:

- Instances have different processing capacities → use Weighted
- Instances are in different geographic zones → use Zone-Aware
- Long-running variable requests → use Least Connections

Q 4 [Tricky] What happens if you use RestTemplate to call 'http://order-service/...' without @LoadBalanced?

■ **ANSWER:**

Without @LoadBalanced, the RestTemplate will try to resolve 'order-service' as a hostname using standard DNS. Since there's no DNS record for 'order-service', the call will fail with:

```
java.net.UnknownHostException: order-service
at java.net.InetAddress.getAllByName(InetAddress.java:1280)
...
Caused by: org.springframework.web.client.ResourceAccessException:
I/O error on GET request for 'http://order-service/api/orders/123'
```

What @LoadBalanced does: It registers a special LoadBalancerInterceptor on the RestTemplate. This interceptor intercepts outgoing calls. When it sees a hostname that doesn't look like an IP, it:

- Treats the hostname as a service ID
- Queries Spring Cloud LoadBalancer: 'resolve order-service'
- LB queries Eureka → returns instances
- LB picks one instance → returns actual IP:port
- Interceptor replaces 'order-service' with actual IP:port in the URL
- RestTemplate makes the actual HTTP call to the real URL

Common mistake: Having two RestTemplate beans, one @LoadBalanced, one plain. Autowiring the wrong one for service-name calls. Always use @Qualifier to be explicit.

Q 5 [Code] Show how to configure per-client load balancing so order-service has one config and payment-service has another.

■ **ANSWER:**

Spring Cloud LB supports per-client configuration via @LoadBalancerClient:

```
// Config for order-service - use default Round Robin
@Configuration
public class OrderServiceLBConfig {
    @Bean
    public ReactorLoadBalancer<ServiceInstance> orderServiceLB(
        ObjectProvider<ServiceInstanceListSupplier> supplier, Environment env) {
        String name = env.getProperty(LoadBalancerClientFactory.PROPERTY_NAME);
        return new RoundRobinLoadBalancer(supplier, name);
    }
}

// Config for payment-service - use Random (illustrative)
@Configuration
```

```

public class PaymentServiceLBConfig {
@Bean
public ReactorLoadBalancer<ServiceInstance> paymentServiceLB(
ObjectProvider<ServiceInstanceListSupplier> supplier, Environment env) {
String name = env.getProperty(LoadBalancerClientFactory.PROPERTY_NAME);
return new RandomLoadBalancer(supplier, name);
}
}

// Register per-service:
@Configuration
@LoadBalancerClient(name='order-service', configuration=OrderServiceLBConfig.class)
@LoadBalancerClient(name='payment-service',
configuration=PaymentServiceLBConfig.class)
public class LoadBalancerConfigs {}

```

Q
6

[Scenario]

Instance 2 of Order Service crashes. The next request is scheduled for Instance 2 (Round Robin). What happens? How does the LB react?

■ **ANSWER:**

This is a real production scenario. Here's the complete failure and recovery sequence:

T=0: Instance 2 crashes. Process terminates. TCP connections closed.

T=0 to T=90s: Instance 2 still in Eureka registry (heartbeat timeout not yet reached).

T=0 to T=30s: Spring Cloud LB client still has Instance 2 in its cached list.

What happens to the unlucky request going to Instance 2:

- Feign/RestTemplate tries to connect to Instance 2's IP:port
- TCP connection refused immediately (process is dead)
- ConnectException is thrown
- If retry is configured: retries on Instance 3 → succeeds
- If no retry: caller gets an error

T=30s: LB cache refreshes from Eureka. Instance 2 still in registry (not evicted yet). Problem continues for another ~60s.

T=90s: Eureka evicts Instance 2 (3 missed heartbeats). Next LB cache refresh: Instance 2 removed. Problem fully resolved.

Best practices to minimize impact:

- Configure Feign/WebClient retry: retry on ConnectException on different instance
- Reduce LB cache refresh to 5s in production
- Use health-check based LB filtering: removes Instance 2 within 25s
- Circuit breaker: stops routing after N failures to detect dead instance faster

Q
7

[Tricky]

Can you use load balancing WITHOUT Eureka? How would you provide the instance list?

■ **ANSWER:**

Yes, Spring Cloud LoadBalancer can work without Eureka. You provide the instance list statically or from another source.

Option 1 — Static Instance List:

```
spring:
cloud:
discovery:
client:
simple:
instances:
order-service:
- uri: http://10.0.1.5:8082
- uri: http://10.0.1.6:8082
- uri: http://10.0.1.7:8082
```

Option 2 — Kubernetes Service Discovery: Use `spring-cloud-kubernetes` instead of Eureka. Kubernetes has built-in service discovery via DNS and its API server.

Option 3 — Consul: Replace Eureka with Consul as the service registry. Spring Cloud Consul integrates with Spring Cloud LB.

Option 4 — Custom `ServiceInstanceListSupplier`: Implement your own supplier that returns instances from any source (database, config file, REST API):

```
@Bean
ServiceInstanceListSupplier customSupplier() {
return new ServiceInstanceListSupplier() {
@Override
public Flux<List<ServiceInstance>> get() {
// Fetch from your custom source
return Flux.just(fetchInstancesFromMyDatabase());
}
@Override public String getServiceId() { return 'order-service'; }
};
}
```

Q
8

[Design]

Compare the full request flow from a mobile client through the Gateway, LoadBalancer, to a microservice and back.

■ ANSWER:

Here is the complete end-to-end request lifecycle with every component involved:

```
MOBILE APP → INTERNET → API GATEWAY → LB → MICROSERVICE → DATABASE
```

Detailed steps:

1. Mobile: GET `https://api.myapp.com/api/orders/123`
+ Authorization: Bearer `eyJhbGc...`
2. API Gateway receives request:
 - a. Global Filter -2 (Auth): validates JWT, extracts `userId='42'`, adds `X-User-Id: 42` header
 - b. Global Filter -1 (Logging): logs incoming request, generates `X-Request-Id: abc123`
 - c. Route matching: `Path=/api/orders/**` → matches `order-service-route`
 - d. Route Filter (CircuitBreaker): checks if circuit is `CLOSED` → yes, proceed
3. Spring Cloud LoadBalancer:
 - a. Sees `uri: lb://order-service`
 - b. Checks local cache: `[10.0.1.5:8082, 10.0.1.6:8082]` (cached from Eureka 15s ago)
 - c. Round Robin picks: `10.0.1.5:8082` (last was `10.0.1.6`)

4. Gateway forwards:

GET http://10.0.1.5:8082/api/orders/123

Headers: Authorization: Bearer..., X-User-Id: 42, X-Request-Id: abc123, X-Source: gateway

5. Order Service processes:

a. @PreAuthorize checks X-User-Id (no JWT validation needed)

b. Queries database: SELECT * FROM orders WHERE id=123

c. Returns 200 OK with OrderDTO JSON

6. Gateway post-processing:

a. Route Filter post (CircuitBreaker): records success

b. Global Filter post (Logging): logs '200 OK in 145ms'

c. Adds response headers: X-Response-Time: 145ms

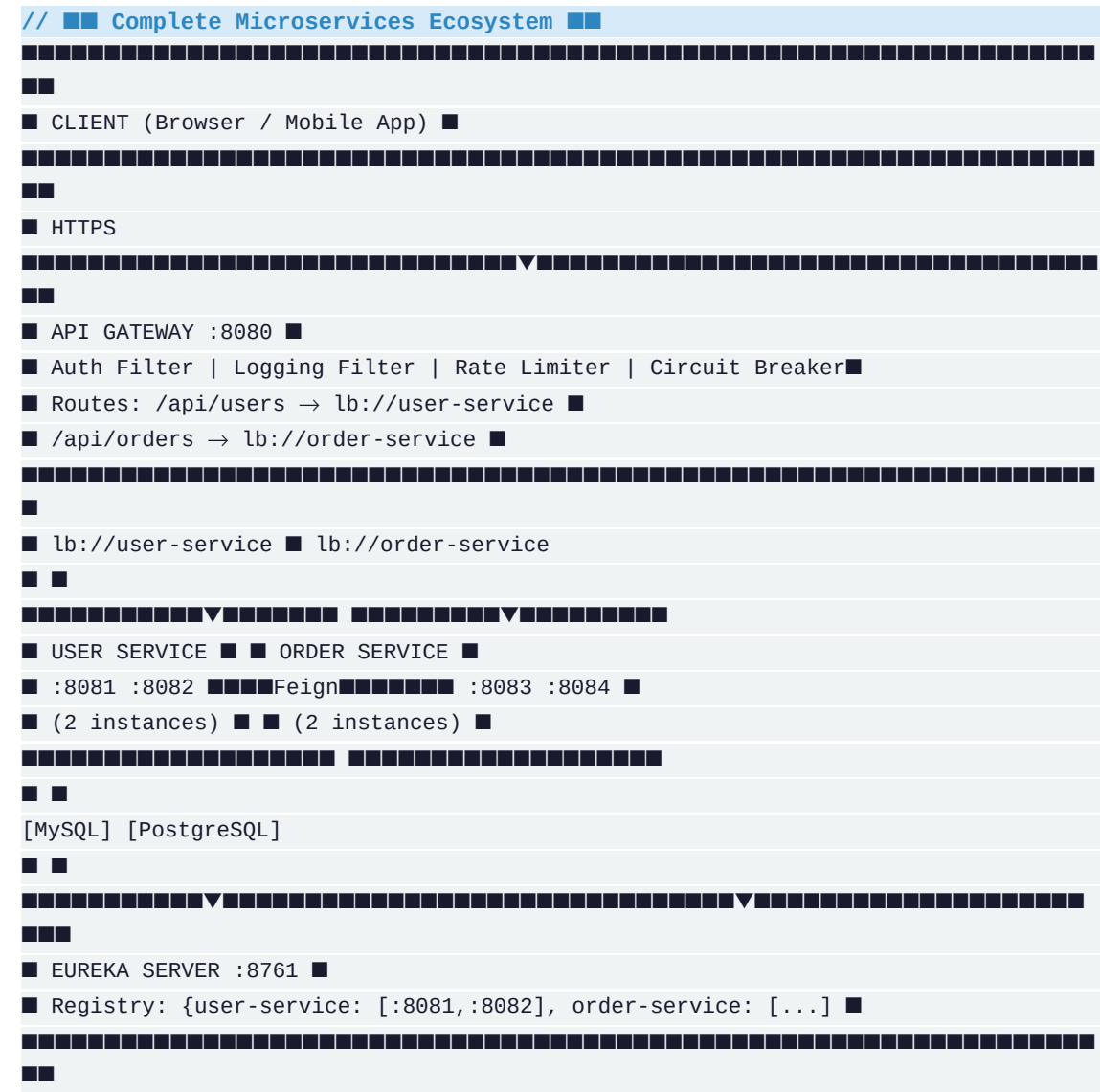
7. Mobile receives:

HTTP 200 OK

+ OrderDTO JSON

Quick Reference — Everything at a Glance

Complete Ecosystem Diagram



Key Annotations Quick Reference

Annotation	Where to Use	Purpose
@EnableEurekaServer	Eureka Server main class only	Activates Eureka Registry and dashboard

Default Port Conventions

Service	Standard Port	Why This Port
Eureka Server	8761	Netflix OSS convention — widely adopted industry standard
Spring Cloud Config Server	8888	Spring Cloud convention
API Gateway	8080	Default HTTP dev port — exposed to clients
Zipkin/Jaeger (Tracing)	9411 / 16686	Tool defaults
Prometheus	9090	Prometheus default
Grafana	3000	Grafana default
Microservices	8081, 8082...	Incremental by convention

★ SENIOR

The journey from fresher to senior engineer in microservices is not about memorizing annotations. It's about deeply understanding WHY each component exists: Eureka exists because IPs are dynamic. Feign exists because REST clients have too much boilerplate. Gateway exists because clients shouldn't know about every service. LoadBalancer exists because one instance is never enough. Master the WHY and the HOW becomes obvious.

This document is a complete self-contained reference for Java Microservices with Spring Cloud. Every concept explained from first principles. Every question answered in depth. Read it as a book, not a cheat sheet. True understanding comes from knowing the problems each tool solves — not just how to configure it.
